Permission Types for Disjointness Reasoning

ANKUSH DAS, Boston University, USA
SHAZ QADEER, Microsoft, USA
GAVIN ROHRER, Boston University, USA

Scalable verification of concurrent heap-manipulating programs requires two kinds of reasoning: *framing*, to preserve facts about one part of the state across updates to another, and *non-interference*, to ensure that concurrently executing threads do not invalidate each other's invariants. Both rely on establishing *disjointness* of program state. To automate this reasoning, we introduce a programming model called NPL that establishes disjointness syntactically, in the program text, rather than in the program logic. NPL replaces the single global heap with a collection of named partial maps keyed by *permissions*: linear values that may move between maps but are never duplicated. Maps and structs can be freely nested, supporting recursive data structures, and permissions are abstract, representing not only memory locations but also thread ids, protocol roles, and rounds. NPL supports fine-grained concurrency through shared mutable global variables, asynchronous threads and atomic functions. A linear-time availability checker guarantees *global disjointness*: in every reachable state, the permissions held by distinct variables are pairwise disjoint. As a result, verification conditions remain classical first-order formulas that SMT solvers discharge automatically. This yields a clean *separation of concerns*: the type system proves disjointness, which logical reasoning then exploits as an assumption, leaving it free to focus on functional correctness. We implemented NPL in the Civl verifier and evaluated it on 44 benchmarks including concurrent data structures, distributed protocols, and parallel GPU-style programs. Verification is fast, taking a median of 1.7 seconds per benchmark, with all but one benchmark verifying in under 10 seconds. Proof overhead is low, averaging 0.87 annotation lines per line of code. Notably, NPL also reduces the size of an existing proof of the Paxos protocol by 33%.

1 Introduction

Verification of functional specifications of concurrent programs has been a long-standing open problem with numerous efforts, both using interactive [23, 39, 40, 57, 58] and solver-based automated techniques [36, 48, 50, 51]. Two closely related challenges lie at its heart: *framing* [61] and *non-interference* [37, 62]. Framing refers to the ability to preserve invariants about one part of the program state across an operation that mutates a different part. Non-interference refers to the ability to reason that concurrently-executing threads do not invalidate each other's invariants. Both challenges share a common foundation: *disjointness* [34, 65]. When the state an invariant depends on is disjoint from the targets of a mutation, framing is immediate. Generalizing to multi-threaded execution, when the state operated on by two threads is disjoint, non-interference follows without further proof obligation. Furthermore, disjointness is not limited to heap reasoning—it is a general principle that applies wherever correctness reduces to proving two values are *distinct*. In GPU programs, for instance, correctness of parallel execution relies on distinct threads operating over disjoint memory regions, thus establishing that thread-level disjointness is central to their verification. Scalable verification of concurrent programs thus depends on effective, automated mechanisms to track and exploit disjointness.

The simplest mechanism for tracking disjointness is *syntactic*: distinct names map to distinct parts of program state, as in Floyd-Hoare logic [21, 33]. If a predicate ϕ depends only on the name x and the contents of a name y distinct from x change due to a statement S, then ϕ can be safely propagated across S. This approach is simple and easy to automate but breaks down for programs that use heap-allocated memory, which inevitably leads to *aliasing* (the possibility of multiple names for the same piece of memory), making syntactic disjointness unsound. Separation

Authors' Contact Information: Ankush Das, Boston University, Boston, Massachusetts, USA; Shaz Qadeer, Microsoft, Redmond, Washington, USA; Gavin Rohrer, Boston University, Boston, Massachusetts, USA.

logic [34, 60, 65] addressed this by internalizing disjointness directly into the program logic via the *separating conjunction*: an assertion $P * Q$ states that P and Q hold over disjoint portions of the heap. This design choice bundles framing and functional correctness together within the logic, where ownership and disjointness are expressed as logical assertions alongside program invariants. The resulting approach is powerful and handles aliasing elegantly, but at the cost of significantly increased logic complexity and difficulty of automation, since every proof obligation must carry explicit heap ownership information.

This paper introduces Nested Permission Language (NPL), a programming model that takes a *separation-of-concerns* approach to the challenges above. Rather than bundling framing and functional correctness together within the program logic as separation logic does, NPL enforces disjointness through the *type system*, leaving the logic free to focus exclusively on functional correctness. NPL is thus a type-directed mechanism for disjointness: the type system guarantees a *global disjointness* invariant across all program state, which the logic then inherits as an assumption. This keeps the underlying logic classical and automation-friendly, since proof obligations carry no explicit ownership information.

NPL supports concurrent programming with dynamically-allocated values. An NPL program may allocate values and bind them to dynamically-generated *tokens*, similar to heap-allocated pointers in traditional languages. However, instead of one implicit global heap, NPL provides numerous *named* heaps present in the program state as local or global variables, modeled as *partial maps* keyed by tokens. These keys are called *permissions*. To model *dynamic ownership*, permissions are allowed to move across distinct partial maps but never duplicated, thereby enforcing *global disjointness*, i.e., partial maps embedded in different parts of program state have disjoint permissions. NPL is flexible and allows partial maps and structs to be mutually *nested*: values inside maps may be structs and fields of structs may be maps, enabling recursively nested data structures like concurrent stacks, linked lists, vectors, and trees that are necessary for practical programming. NPL also supports fine-grained concurrency with statement-level preemption and *atomic functions* to mark operations that execute without interleaving from other threads.

NPL embeds a *linear type system* [24, 25] that tracks *availability* of permissions and prevents duplication. To keep permission tracking feasible, NPL restricts how partial maps and structs may be accessed: partial maps only support *insert* and *delete* (direct reads and writes are forbidden), and structs must be explicitly packed and unpacked to access their fields. Function calls pass arguments and results by value, transferring the permissions of the arguments to the callee and those of the results back to the caller. Together with the convention that all global variables are available at function boundaries, this enables modular availability checking without inspecting callee bodies. This careful design ensures that type checking runs in *linear time*.

Crucially, NPL's notion of disjointness generalizes *beyond* memory locations. Permissions in NPL are an abstract concept: they can represent heap locations, but equally well thread identifiers, protocol roles, or round numbers in a distributed protocol. In GPU programs, thread-level disjointness ensures that parallel threads operate on non-overlapping data, enabling interference-free reasoning about parallel execution. In protocol verification, such as the Paxos consensus protocol, disjointness of permissions associated with rounds and acceptors enables modular commutativity arguments that are central to correctness proofs. This generality makes NPL broadly applicable as a mechanism for disjointness-based reasoning wherever correctness depends on showing that distinct entities do not interfere.

Our goal is to demonstrate that NPL aids scalable verification of realistic concurrent programs. To this end, we have implemented NPL in the Civl verifier [44, 45] and evaluated it on 44 benchmarks spanning lock implementations, concurrent data structures (e.g., Treiber stack, Michael-Scott queue, and lock-coupling list), distributed protocols (e.g., Paxos, ABD, and two-phase commit), and parallel

GPU-style programs (e.g., prefix scan and parallel reduction). Verification is fast, taking a median of 1.7 seconds per benchmark, with all but one benchmark verifying in under 10 seconds, which makes interactive program development feasible. Proof overhead is low, averaging 0.87 annotation lines per line of code. Compared to Raven [28], a verifier that also targets SMT solvers but is based on concurrent separation logic, NPL has lower proof overhead on all seven benchmarks common to both (0.68 vs 1.65 on average), with comparable verification times. NPL has also enabled the construction of verified programs that mix data structures and fine-grained concurrency, and has reduced the size of an existing proof of the Paxos protocol by 33%.

To summarize, NPL makes the following contributions:

- achieves type-directed disjointness by enforcing a global disjointness invariant through a linear type system (Sec 6), thereby separating framing from functional correctness
- enables ergonomic and automated functional verification of nested data structures with fine-grained concurrency (Sec 6.1),
- generalizes disjointness reasoning beyond heap locations, as demonstrated by verified case studies including the Paxos consensus protocol and GPU programs (Sec 7),
- demonstrates fast verification with low proof overhead on 44 benchmarks, outperforming Raven in proof overhead on all common benchmarks (Sec 7).

The supplement contains the entire formalization of NPL.

2 The NPL Programming Model

To set the stage for NPL, we first recall how the classic Reynolds' in-place list reversal [65] is analyzed using separation logic with the goal of proving *memory safety*. As a first step, we introduce a recursive predicate $\mathbf{list}(x)$ to represent a well-formed singly-linked list rooted at x, defined as:

$$\mathbf{list}(x) \triangleq (x = \mathsf{nil}) \vee (\exists v, n.\ x \mapsto (v, n) * \mathbf{list}(n))$$

where $x \mapsto (v, n)$ asserts that location x holds data v and next pointer n. Central to reasoning is the *separating conjunction* $P * Q$ modeling that predicates P and Q hold over *disjoint* portions of the heap. The above predicate defines that either *(i)* x is an empty list, or *(ii)* there exist v and n such that x holds data v and its next pointer is n, and n is a well-formed singly linked list. Moreover, the $*$ operator in the predicate describes that the contents of n are disjoint from x. Thus, the predicate encodes both the contents of each heap portion and the disjointness between them.

To show a direct contrast with NPL[1], we describe a recursive implementation of list reversal implemented by `rev_append` , which reverses the list at `x` and prepends it to `acc`. The function returns `new_acc` that denotes the result of the function call. We annotate the program with its specification and intermediate assertions (in dark green) at each program point:

```
struct List { data: int, next: List* }
fn rev_append(x: List*, acc: List*) ⟶ new_acc: List* {
  { list(x) * list(acc) }
  if x = nil then  { new_acc := acc }   // x is empty  ⇒  return acc
  else                                  // x is not empty
    { ∃v, n. x ↦ (v, n) * list(n) * list(acc) }   // unfold list(x)
    t := x⟶next                         // assign x's tail to t
    { ∃v. x ↦ (v, t) * list(t) * list(acc) }      // instantiate n with t
    x⟶next := acc                       // set x's tail to acc
    { ∃v. x ↦ (v, acc) * list(t) * list(acc) ⇒ list(t) * list(x) }
```

[1]NPL is based on recursion, not iteration: this is not a fundamental limitation, however, as iteration can be modeled in NPL

```
    new_acc := rev_append(t, x) }       // t becomes new x, and x becomes new acc
```

The critical step is the write '$\mathsf{x} \longrightarrow \mathsf{next} := \mathsf{acc}$'. The pre-condition to the write is '$\mathsf{x} \mapsto (v, \mathsf{t}) * \mathbf{list}(\mathsf{t})$' guaranteeing that x and the locations storing the elements of t are *disjoint*. Updating $\mathsf{x} \longrightarrow \mathsf{next}$ modifies only the cell at x, so $\mathbf{list}(t)$ survives intact by the *frame rule*:

$$\frac{\{P\}\ C\ \{Q\}}{\{P * R\}\ C\ \{Q * R\}} \quad \text{provided } C \text{ does not modify the footprint of } R$$

Here $R = \mathbf{list}(\mathsf{t})$ and the disjointness encoded in $*$ ensures C leaves R intact. The recursive call is valid because the predicate $\mathbf{list}(\mathsf{x})$ is implied by $\exists v.\ \mathsf{x} \mapsto (v, \mathsf{acc}) * \mathbf{list}(\mathsf{acc})$:

$$\begin{array}{lll} \exists v.\ \mathsf{x} \mapsto (v, \mathsf{acc}) * \mathbf{list}(\mathsf{acc}) & \Rightarrow & \exists v, n.\ \mathsf{x} \mapsto (v, n) * \mathbf{list}(n) \qquad \text{(instantiate } n := \mathsf{acc}\text{)} \\ & \Rightarrow & (\mathsf{x} = \mathsf{nil}) \vee (\exists v, n.\ \mathsf{x} \mapsto (v, n) * \mathbf{list}(n)) \qquad \text{(weaken)} \\ & \Leftrightarrow & \mathbf{list}(\mathsf{x}) \qquad \text{(fold definition)} \end{array}$$

And thus, the post-condition after the write implies $\mathbf{list}(\mathsf{t}) * \mathbf{list}(\mathsf{x})$ validating the recursive call.

The main takeaway here is that the logical reasoning engine does double duty: it simultaneously expresses functional content (the heap contents are correct) and structural content (the two heap regions are disjoint). As a result, disjointness is not an assumption the logic inherits but a claim that must be re-established at every step, and every proof obligation must carry explicit heap ownership.

2.1 List Reversal in NPL

We now show how NPL offloads this structural reasoning to the type system, freeing the logic to focus purely on functional content. We begin by redefining the type of an integer linked list in NPL:

```
struct List {
  data: int,                        // integer stored in list element
  next: Option<Tok>,                // optional location: None if last element
  lheap: Map<One<Tok>, List> }      // mapping from location to next element
```

NPL introduces a builtin type called `Tok` that represents a *globally unique token*. For the example above, this is used to model a memory location, but it can be used to model thread ids, or round number in a protocol. The field `data` stores the integer element held in each list entry, exactly as in a standard linked list. The field `next` stores an optional *location* of type `Tok` that identifies the remainder of the list. The field `lheap` is a map of type `Map<One<Tok>, List>` that binds the location named in `next` to the actual next `List` value. We traverse a value `l0` of type `List` as follows. If `l0.next` is `None`, the list has ended. Otherwise `l0.next` is `Some(loc)`, and we obtain the next list value `l1` by looking up `loc` in `l0.lheap`. We continue in the same way by examining `l1`, until we reach the end of the list.

Why is the key type of `lheap` `One<Tok>` *rather than* `Tok`? In NPL, a value of type $\mathrm{One}\langle\tau\rangle$ models an *abstract permission* and must therefore be treated *linearly*. In our example, `lheap` is thus a linear map. The permissions held by a linear map are defined recursively as its set of *keys* together with the permissions stored in each of its values. At any program point, the total multiset of permissions in the program is the union of the permissions held by all linear variables in scope, whether *local or global*. NPL guarantees that ***these permissions are all mutually disjoint***, so the multiset has no duplicates and is in fact a set. For the value `l0` above, this means that `keys(l0.lheap)`, `keys(l1.lheap)`, ... are all mutually disjoint.

This notion of disjointness is exactly what is captured by the recursive separation logic predicate $\mathbf{list}(x)$. But, NPL's type system is able to guarantee this property automatically with *linear-time*

checking and with a modest syntactic overhead. This, in turn, also dramatically simplifies logical reasoning which now primarily focuses on functional correctness.

How does disjointness assist program verification? As a first example, suppose we wish to prove that the list above is acyclic. Formally, for a list `[l0, l1, ...]`, we must establish that `l0.next`, `l1.next`, ... are all pairwise distinct. Because this property relies on deeply embedded data, it is difficult to express, let alone prove, purely logically.

In NPL, this can be achieved much more directly. Essentially, the `List` data structure needs to maintain a crucial *invariant*:

- either `next` is `None`,
- or `next` is `Some(loc)` and `loc` is a valid key in `lheap` mapping to the next `List` element.

This can be modeled as the following predicate:

$$\mathbf{list}_{\mathsf{NPL}}(\mathsf{x}) \triangleq (\mathsf{x.next} = \mathsf{None}) \vee (\exists \mathsf{loc}.\ \mathsf{x.next} = \mathsf{Some}(\mathsf{loc}) \wedge \mathsf{loc} \in \mathbf{keys}(\mathsf{x.lheap}))$$

We *associate* the predicate $\mathbf{list}_{\mathsf{NPL}}$ with the type `List`, so that it is assumed to hold of every value in scope of that type at every program point. This is precisely where the type system and logical reasoning *interact*. The logical engine is responsible for maintaining $\mathbf{list}_{\mathsf{NPL}}$ as a local invariant on each `List` value, a purely functional obligation about a single node and the keys of its own `lheap`. The type system independently guarantees that permissions held across all linear maps are mutually disjoint. Neither fact is sufficient on its own, but together they let us derive global structural properties of the heap from a local and easily checkable invariant. This interplay yields the acyclic property for free! The invariant $\mathbf{list}_{\mathsf{NPL}}$ ensures that each `next` field is a key in the corresponding `lheap`, and disjointness ensures that all the `lheaps` are mutually disjoint, which together imply that the `next` fields are distinct.

Unlike **list**, the invariant $\mathbf{list}_{\mathsf{NPL}}$ is *non-recursive* and only represents the functional content of the invariant. Further, this functional content can be established *purely locally* at each list element. The recursion has been reduced entirely to the type system, and the recursive structure of `List` (along with the local $\mathbf{list}_{\mathsf{NPL}}$ invariant) is sufficient in establishing global disjointness. This locality is what makes NPL programs generally far more amenable to *automated* verification.

This property naturally generalizes to multiple lists. Suppose we have two distinct variables `l` and `m` of type `List`, and we collect their elements as `l0`, `l1`, ... and `m0`, `m1`, ... respectively. Then NPL enforces *global disjointness*, so that `l0.next`, `l1.next`, ... and `m0.next`, `m1.next`, ... are all pairwise distinct whenever they satisfy ϕ. This recovers the classical *syntactic framing* property, again without appealing to recursion in the program logic.

Linear maps are a simple yet powerful foundation for expressing and enforcing disjointness. Disjointness is expressed by collecting the keys stored across *all* linear maps and requiring them to be pairwise distinct, that is, *globally unique*. Disjointness is enforced by our *availability checker*, a component of the type checker whose single guiding principle is to *prevent cloning of linear values*. The type system distinguishes ordinary from linear variables and tracks the *availability* of the latter. Concretely, after an assignment $y \leftarrow x$, the variable x becomes unavailable and y becomes available. An unavailable variable is treated like an ordinary variable, with no linear operations permitted on it, which guarantees that every permission is reachable through exactly one linear variable at any program point. We formalize the type system and prove it sound in Section 6.

Reverse Function. We now rewrite the in-place list reversal function in NPL. Before describing the pre/post-conditions, we first explain the NPL syntax. List reversal is implemented using `rev_append` as follows (linear variables marked in red):

```
1 fn rev_append(x: List, acc: List) ⟶ new_acc: List {
2   { list_NPL(x) ∧ list_NPL(acc) }
```

```
3   List { data, next, lheap } = x;          // unpack x into its fields
4   match next {
5     None ⇒ { new_acc = acc }               // list reversed; return acc
6     Some(loc) ⇒ {
7       (loc_p, t) = lheap.del(loc);         // retrieve next element t from lheap
8       { list_NPL(t) ∧ list_NPL(acc) ∧ lheap = empty }
9       next = Some(loc_p.val);              // set next to loc_p's value
10      lheap.ins(loc_p, acc);               // reuse loc_p: now points to acc
11      { list_NPL(t) ∧ list_NPL(acc) ∧ next = Some(loc) ∧ loc ∈ keys(lheap) }
12      x = List { data, next, lheap };      // pack list x's contents back
13      { list_NPL(t) ∧ list_NPL(x) }
14      new_acc = rev_append(t, x); } } }    // recursive call
```

The `rev_append` function takes two list arguments: an input list `x` and an accumulator `acc`. Like before, it reverses `x` and prepends the result onto `acc`, returning `new_acc`. The function body is defined as follows. First, the input list `x` is unpacked into its fields `data`, `next`, and `lheap` (line 3). We then pattern match on `next` to determine whether `x` is exhausted. If `next` is `None`, we have reached the end, so there is nothing left to reverse and `acc` is left unchanged and returned. Otherwise `next` is `Some(loc)`, and we detach the tail by removing the entry at `loc` from `lheap` using `lheap.del(loc)`, which returns the freed permission `loc_p` and the tail list `t`. We update `next` to `Some(loc_p.val)` (line 9) and splice the current node onto `acc` by reinserting the same permission with `lheap.ins(loc_p, acc)` (line 10), so that the reused location now maps to `acc`. The notation `.val` returns the value stored inside $\mathrm{One}\langle\tau\rangle$. The assignment `x = List { data, next, lheap }` sets `x` as the new accumulator. Finally, we recurse on the tail `t` with the new accumulator, reversing the remainder of `x` onto it.

NPL provides no primitives for directly reading the values of maps or structs. To access a struct, one must unpack it, read the relevant field, and pack it back; to access a map entry, one must delete it and later insert it back. This discipline makes permission tracking straightforward, since every linear operation is essentially a *permission transfer*. Unpacking a struct transfers its permissions to its fields and packing transfers them back, just as an assignment moves permissions from one variable to another. Likewise, a map delete transfers permissions from the entry to its destination, and a map insert transfers them back. This adds only a modest syntactic overhead while keeping the availability checker *linear-time* and modular.

Reasoning about Well-Formedness. The memory safety of `rev_append` can also be proved by reusing the predicate $\mathbf{list}_{\mathrm{NPL}}$, the functional component of the separation-logic predicate **list**. We revisit the recursive function focusing on predicates (in dark green); the precondition and postcondition are the same: $\mathbf{list}_{\mathrm{NPL}}(\mathtt{x}) \wedge \mathbf{list}_{\mathrm{NPL}}(\mathtt{acc})$.

We trace how the logic discharges the intermediate assertions of the `Some` branch. Unpacking `x` exposes its fields `data`, `next`, and `lheap`, and the precondition $\mathbf{list}_{\mathrm{NPL}}(\mathtt{x})$ tells us that `next` is `Some(loc)` and `loc` is the unique key of `lheap`. The delete `lheap.del(loc)` therefore empties `lheap`, moves its sole permission to `loc_p`, and binds `t` to the value that was stored at `loc`. Because that value is itself a well-formed list, the logic concludes $\mathbf{list}_{\mathrm{NPL}}(\mathtt{t})$, and since the operation never touched `acc`, the assertion $\mathbf{list}_{\mathrm{NPL}}(\mathtt{t}) \wedge \mathbf{list}_{\mathrm{NPL}}(\mathtt{acc}) \wedge \mathtt{lheap} = \mathtt{empty}$ holds.

The next three statements rebuild a node around `acc`. The insert `lheap.ins(loc_p, acc)` places `acc` at key `loc_p.val`, the assignment `next = Some(loc_p.val)` points `next` at exactly that key, and the pack `x = List { data, next, lheap }` reassembles these fields into `x`. The resulting node has its `next` field equal to the unique key of its `lheap`, which is precisely $\mathbf{list}_{\mathrm{NPL}}(\mathtt{x})$, so the

assertion $\mathbf{list}_{\mathsf{NPL}}(\mathtt{t}) \wedge \mathbf{list}_{\mathsf{NPL}}(\mathtt{x})$ holds. The logic establishes this from purely local facts about `loc_p`, `next`, and `lheap`, with no reasoning about `t` or any other part of the program state.

This locality is the payoff of separating concerns. In separation logic, write '`x → next := acc`' forces the frame rule to re-certify that `t` is disjoint from the modified cell, a fact threaded explicitly through the separating conjunction at every step. In NPL, this disjointness is never a proof obligation: the type system has already guaranteed that the permissions in `t`, `x`, and `acc` are globally disjoint, so the logic may simply assume that $\mathbf{list}_{\mathsf{NPL}}(\mathtt{t})$ survives the write. In summary, the type system handles framing, while the logic reasons purely about functional content.

2.2 Doubly Linked Lists: Rust vs NPL

At first, it might seem that NPL's linear type system, and therefore, its expressivity is a strict subset of Rust. However, this is not the case! We highlight the distinction between Rust and NPL using a a canonical example: doubly linked list. For this data structure, each node holds a reference to both its successor (`next`) and its predecessor (`prev`). A natural attempt to define a doubly linked list in Rust is to use the `Box<T>` smart pointer:

```
struct DList {
    data: i32,                          // integer stored in list element
    next: Option<Box<DList>>,           // pointer to next element
    prev: Option<Box<DList>>, }         // pointer to previous element
```

In Rust, `Box<T>` heap-allocates a value of type `T` and *owns* it: when the `Box` is dropped, the allocation is freed. The type above therefore asserts that each node simultaneously owns its successor (via `next`) and its predecessor (via `prev`). But ownership in Rust is *exclusive*: a value may have at most one owner at any time. An interior node would need to be owned by two distinct `Box` values, namely the predecessor's `next` and the successor's `prev`, which is impossible.

Even a two-node list immediately exposes this problem:

```
let node2 = Box::new(DList { data: 2, next: None, prev: None });
let mut node1 = Box::new(DList { data: 1, next: Some(node2), prev: None });
node1.next.as_mut().unwrap().prev = Some(node1); // error: use of moved value
```

The Rust compiler rejects the last line: `node1` was moved when constructing `node1.next`, so it is no longer available to place into `prev`. There is no way to form the chain of `Box` pointers that a doubly linked list requires.

The root cause is that in Rust, *all heap references are owning by default*. Non-owning references (borrows `&T` or `&mut T`) carry a *lifetime* that the borrow checker must verify does not outlive the owner. For doubly linked lists, the `prev` and `next` lifetimes are mutually dependent, forming a cycle the borrow checker cannot resolve statically. In practice, doubly linked lists in Rust therefore require either runtime-checked shared ownership (e.g., `Rc<RefCell<DList>>`) with explicit cycle-breaking via `Weak` pointers or *unsafe* raw pointers, both of which sacrifice the static guarantees Rust's type system is designed to provide.

Doubly Linked Lists in NPL. NPL resolves this tension through a fundamental distinction between two kinds of token:

- **`One<Tok>`**: a *linear* value that carries *ownership of a permission*. The availability checker ensures each **`One<Tok>`** resides in exactly one variable at any program point, enforcing the global disjointness invariant.

- **`Tok`**: an *ordinary* value that records the *identity* of a permission without claiming ownership of it. A **`Tok`** may be freely copied and read, and, like all ordinary values in NPL, does not participate in the disjointness invariant.

Using this distinction, a doubly linked list in NPL separates node ownership from navigation:

```
struct Node { data: int, next: Option<Tok>, prev: Option<Tok> }
struct DList {
  head: Option<Tok>,              // identity of the first node
  tail: Option<Tok>,              // identity of the last node
  nodes: Map<One<Tok>, Node> }    // linear map: owns all nodes
```

Ownership of every node is concentrated in the single linear map `nodes`, whose **`One`**`<`**`Tok`**`>` keys are tracked by the availability checker. The global disjointness invariant therefore applies to `nodes` alone: all nodes in a `DList` value must be globally unique and no two `DList` values in scope can own the same node. Navigation is encoded directly inside `Node`: the `next` and `prev` fields are plain **`Tok`** values recording the identity of neighboring nodes without owning them. Since **`Tok`** is ordinary, these fields may be freely read and copied without any interaction with the availability checker. To traverse the list, one starts from `head` and follows `next` fields, looking up each successor identity in `nodes`. Dually, one can start from the `tail` and follow the `prev` fields to reach the `head`. Modifying a node requires deleting its entry from `nodes` to obtain the **`One`**`<`**`Tok`**`>` permission and the `Node` value, updating the fields, and reinserting.

This example illustrates a broader design principle of NPL. NPL integrates two layers: a *linear layer* of **`One`**`<`**`Tok`**`>` values governed by the availability checker, and an *ordinary layer* of plain values (including **`Tok`**) untouched by the linear type system. These layers interact at precisely controlled points, namely map insertion and deletion and struct packing and unpacking, where permissions transfer between them. The result is a type system that is simultaneously expressive, handling data structures like doubly linked lists, yet simple enough for linear-time checking, without any lifetime tracking. This simplicity is possible because NPL does not task its type system with guaranteeing memory safety; that obligation is delegated to the logical engine. The type system need only track disjointness, which the logical engine then exploits to establish functional correctness. Finally, there are no special requirements on the logical engine; any verifier capable of reasoning with Floyd-Hoare triples is adequate for the sequential fragment of NPL.

3 Concurrency in NPL

A central advantage of NPL's programming model is that its two organizing principles, linear maps and global disjointness, extend smoothly from the sequential setting to concurrent programs, without requiring separate concurrency-specific reasoning machinery. This section substantiates this claim with a barrier synchronization protocol. Before presenting the protocol, we set the context by describing NPL's concurrent programming and execution model. A program consists of a fixed set of global variables together with two kinds of functions, *ordinary* and *atomic*, the latter distinguished syntactically by an **`atomic`** qualifier. Global variables may only be accessed from within atomic functions; ordinary functions are restricted to local state and must delegate any access to shared state to an atomic call. Threads are created by spawning a function *asynchronously*, after which it executes concurrently with its parent. Execution *interleaves arbitrarily*, i.e., a thread yields control at any program point, except for atomic functions, which run to completion as an atomic block without interruption once entered. This restriction on where yields may occur induces a corresponding restriction on the call graph: ordinary functions may call both ordinary and atomic

functions, but atomic functions may only call other atomic functions, so that a thread already inside an atomic region can never yield partway through.

NPL's availability checker generalizes just as smoothly to concurrent programs. Since ordinary functions cannot access global variables, threads executing them can interleave arbitrarily without any risk of unsoundness, as their behavior can only depend on local state. Such functions also trivially preserve the availability of every global, imposing no extra proof obligation on the availability checker. The real work falls on atomic functions: every global variable is assumed available at the start of an atomic function body and must be shown available again at every return point. This assumption is what makes availability checking *scale*: because every global is guaranteed available at every function boundary, the checker never needs to track availability across a function call, atomic or not, and so can check each function body in isolation using only local reasoning. As a result, availability checking remains *linear* in the size of the program, even in the presence of concurrency.

Barrier Synchronization. In this protocol, a dedicated *collector* thread coordinates a stop-the-world operation over a fixed pool of N *mutator* threads. The collector periodically needs exclusive access to shared memory (e.g., to scan and reclaim unreachable objects) and to do so safely, it must first wait for all N mutators to park. Each mutator, meanwhile, runs freely and only needs to check, at safe points in its execution, whether the collector has requested a park, in which case it parks itself until the collector signals that it is done and releases the mutators to resume running.

To model the barrier, we introduce a type called `Role` and three global variables:

```
union Role { Left, Right }
global barrier_on: bool
global unparked: int
global parked: Set<One<Tag<Role>>>
```

A `Role` value is just a bare label, either `Left` or `Right`. To bundle tokens with an arbitrary type, NPL provides a built-in struct type `Tag<T>` defined as follows[2]:

```
  struct Tag<T> { tok: Tok, tag: T }
```

A `Tag<T>` pairs a `Tok` (its `tok` field) with a value of an arbitrary type `T` (its `tag` field), and wrapping it in `One<Tag<T>>` makes that specific pairing a linear permission. For this example, we instantiate `T` with `Role`: a `Tag<Role>` permission carries both a `Tok` (its `tok` field) identifying *which* mutator it belongs to and a `Role` (its `tag` field) recording *which* of the two permissions it is. Every mutator is spawned holding *two* linear permissions sharing their token (in `tok` field), `left: One<Tag<Role>>` and `right: One<Tag<Role>>` (tagged `Left` and `Right` respectively), both minted together at spawn time. The `left` permission is the mutator's *ticket* into the barrier: it represents the mutator's *participation*, i.e., proof that it is one of the N threads the collector must wait for, and it moves back and forth between the mutator and the global `parked` set. NPL's global disjointness is key here: proving the protocol correct amounts to showing that once the collector observes every ticket accounted for, it has genuinely heard from all N mutators, and not, say, from the same mutator's ticket *counted twice* (the latter scenario being eliminated because permissioned tickets cannot be cloned). The `right` permission, in contrast, is never relinquished: it stays with the mutator for its entire lifetime and acts as a *capability*, required to reclaim `left` once it has been handed over. Since a mutator's ticket is deposited into the shared `parked` set while parked, we need a mechanism for the mutator to later reclaim *its own* ticket and ensure it does not reclaim another mutator's ticket. The `right` permission serves exactly that purpose: because it is linear and never shared, only the

[2]the purpose of `Tag` will be clarified further when discussing creation of the mutator threads

mutator holding it can ever reclaim the corresponding ticket. This lets NPL rule out two threads racing to reclaim the same parked ticket *by typing alone*, rather than by logical reasoning.

The flag `barrier_on` records whether the collector currently wants mutators parked; `unparked` counts how many of the N mutators have yet to park; and `parked` is a linear set collecting the `Left` tickets of mutators that have already done so. The type `Set<T>` can be interpreted simply as a type alias for `Map<T, unit>`, and therefore, just like linear maps, serves as the simplest foundation for storing linear permissions.

The mutator's protocol is implemented as follows:

```
atomic fn is_barrier_on() → b: bool { b := barrier_on }

atomic fn enter_barrier(left: One<Tag<Role>>) ensures MutatorInv(left) {
  parked.ins(left);                    // hand my ticket to the collector
  unparked := unparked - 1 }

atomic fn try_leave_barrier(right: One<Tag<Role>>)
    → (attempt: Option<One<Tag<Role>>>, right: One<Tag<Role>>)
    requires MutatorInv(left) {
  if is_barrier_on() then
    attempt := None
  else {
    left = parked.del(Tag { tok = right.val.tok, tag = Left });  // reclaim ticket
    unparked := unparked + 1;
    attempt := Some(left) } }

fn wait_for_release(right: One<Tag<Role>>)
    → (left: One<Tag<Role>>, right: One<Tag<Role>>) {
  (attempt, right) := try_leave_barrier(right);
  match attempt {
    None ⇒ (left, right) := wait_for_release(right)    // yield and poll again
    Some(l) ⇒ left := l } }

fn mutator(left: One<Tag<Role>>, right: One<Tag<Role>>) {
  if is_barrier_on() then {
    enter_barrier(left);
    { MutatorInv(left) }
    (left, right) := wait_for_release(right) }
  // safe to access shared memory here
  mutator(left, right); }
```

To park, a mutator calls `enter_barrier`, which consumes its ticket `left` by inserting it into the global `parked` set and decrements `unparked`. Having given up `left`, the mutator can no longer touch shared memory, but it still holds `right`, which is now its sole means of getting `left` back. The mutator then calls `wait_for_release`, threading `right` through, which calls `try_leave_barrier`. In a *single atomic step*, `try_leave_barrier` checks `barrier_on` and, only if it is currently false, reads `right`'s `tok` field to reconstruct the matching `Left`-tagged permission and looks it up in

parked to reclaim exactly the ticket the mutator deposited, increments unparked back, and returns Some(left); otherwise it returns None and wait_for_release recurses, yielding and polling again. The top-level mutator function ties these together: it checks if barrier is on, and only if true, does the mutator park and wait. Otherwise, and in all cases once wait_for_release returns, the mutator is holding left again and may safely proceed to access shared memory.

The collector's protocol is defined as follows:

```
atomic fn set_barrier(b: bool) { barrier_on := b }

atomic fn all_parked() ⟶ b: bool { b := (unparked = 0) }

fn wait_for_all_parked() {
  if !all_parked() then wait_for_all_parked() }   // yield and poll again

fn collector() {
  set_barrier(true);
  wait_for_all_parked();
  // exclusive access to shared memory here: every mutator is parked
  set_barrier(false);
  collector() }
```

The collector first calls set_barrier(true), announcing it wants every mutator parked; any mutator that subsequently calls is_barrier_on will enter the barrier. It then calls wait_for_all_parked, which, just like the mutator's own poll loop, recurses and yields until all_parked observes unparked = 0, i.e., until every mutator still running has deposited its ticket into parked. Only at that point does the collector perform its exclusive work; it then calls set_barrier(false), clearing the flag so that every parked mutator can reclaim their ticket and resume.

Verification and Yield Invariants. To verify concurrent programs, NPL supports a special class of invariants called yield invariants [43, 45] that are meant to be preserved across *all* yield-to-yield fragments. For this protocol, correctness rests on a single global yield invariant:

$$\textbf{BarrierInv} \triangleq |\mathsf{parked}| + \mathsf{unparked} = N$$

Since regular functions cannot access globals, this invariant must be preserved by *every* atomic function. Concretely, this invariant is assumed on entry to every atomic function and must be reestablished on exit, so it holds at every yield point. We now show how **BarrierInv** is preserved by each atomic function.

How do permissions assist in verification? Naively, it may seem straightforward to check that each atomic function preserves **BarrierInv**. For instance, consider enter_barrier that inserts left into |parked| and decrements unparked by one. How do we know that left does not already exist in parked? This is exactly where disjointness plays a crucial role: since left has type **One**<**Tag**<Role>>, we are guaranteed that there is no clone of it in the program state, particularly inside parked. Thus, insertion will certainly increment the size of parked.

The reasoning for try_leave_barrier is slightly more complicated and requires careful coordination between logical reasoning and the type system. In the **if** branch, both parked and unparked are unmodified, so **BarrierInv** is trivially preseved. In the **else** branch, preserving the invariant requires that the reconstructed permission **Tag** { tok = right.val.tok, tag = Left } must already exist in parked; otherwise, the delete would fail and the invariant would be violated.

This is precisely the role of a second yield invariant defined for a specific mutator's ticket:

$$\textbf{MutatorInv}(\text{left}) \triangleq \text{left} \in \text{parked}$$

The `enter_barrier` function *establishes* **MutatorInv**(left) (hence the *ensures* **MutatorInv**(left)) because it inserts `left` into `parked`. Conversely, `try_leave_barrier` *requires* **MutatorInv**(left) on its reclaiming branch: this is exactly the fact needed to justify that its call to `parked.del` succeeds, since `right.val.tok` is the same token that `left` carries, so `Tag {tok = right.val.tok, tag = Left}` is definitionally the same ticket that **MutatorInv**(left) refers to. No other atomic function ever inserts or removes this ticket, so once `enter_barrier` establishes **MutatorInv**(left) for this mutator, it remains true at every yield point, regardless of how many other threads run in between, until the same mutator's `try_leave_barrier` consumes it. This is why each mutator retains `right` for the entire protocol: it is the only capability on the thread's identity needed to both state and later discharge **MutatorInv**.

Why does **BarrierInv** *suffice to prove the collector safe?* When the collector observes `unparked = 0` in `all_parked`, **BarrierInv** tells us that $|\text{parked}| = N$. This is the final place where global disjointness proves useful: because `parked` is a linear set, NPL guarantees that its N elements are pairwise distinct `Tag<Role>` permissions, and must therefore contain tickets from *all* mutators; there is no way for the same mutator's ticket to be counted twice, and no room left for any ticket to be missing. Consequently, at the instant the collector observes `unparked = 0`, every mutator currently lacks its `left` permission, so none of them can be touching shared memory making it safe for the collector to perform a scan.

Spawning the Mutators. It remains to show how the N mutator threads referenced throughout this section are created in the first place via NPL's `async` function:

```
fn allocate(n: int) {
  if n > 0 then {
    roles = Set<Role>();                // initialize ordinary roles: Set<Role>
    roles.ins(Left);
    roles.ins(Right);                   // roles contains both variants
    (slots, new_one_tok) = new roles;   // slots: Set<One<Tag<Role>>> is linear
    new_tok = new_one_tok.val;          // new_one_tok: One<Tok>, new_tok: Tok
    left = slots.del(Tag { tok = new_tok, tag = Left });
    right = slots.del(Tag { tok = new_tok, tag = Right });
    async mutator(left, right);
    allocate(n-1) } }
```

Each call to `allocate` first builds the ordinary set `roles = {Left, Right}` by inserting both `Role` labels one at a time, then passes it to `new`, which mints a single fresh token `new_one_tok` together with the linear set `slots`, containing exactly one `Tag<Role>` per element of `roles`, i.e., a `Left`-tagged and a `Right`-tagged permission, both sharing that same fresh token. Reading `new_tok`, the ordinary `Tok` off `new_one_tok`, it then removes each of the two permissions from `slots` by name, one tagged `Left` and one tagged `Right`, both bundled with `new_tok`, binding them to `left` and `right` and handing them directly to a newly spawned `mutator` thread. Because `new` mints a genuinely fresh token on every call, no two calls to `allocate` can ever produce the same `new_tok`, so no two mutators can ever end up sharing a token. This is exactly the disjointness fact that **BarrierInv** and **MutatorInv** both rest on. The `allocate` function then recurses on `n-1`, so calling `allocate(N)` during initialization will spawn the N mutator threads.

$$\text{Types} \quad \tau ::= \mathsf{One}\langle\tau\rangle \mid \mathsf{Map}\langle\tau,\tau\rangle \mid \mathsf{Tag}\langle\tau\rangle \mid \mathsf{Opt}\langle\tau\rangle \mid \mathsf{Tok} \mid T \mid \mathsf{unit} \mid \mathsf{bool} \mid \mathsf{int}$$

$$\text{Values} \quad v ::= \mathit{One}(v) \mid \mathit{Map}\{\overline{(v,v)}\} \mid \langle T\rangle\{\overline{g : v}\} \mid \langle T\rangle g(v) \mid l \mid \bullet \mid \mathbb{B} \mid \mathbb{Z}$$

$$\begin{aligned}\text{Expr} \quad e ::= {} & \langle T\rangle\{\overline{g : e}\} \mid e.g \mid \langle T\rangle g(e) \mid e \text{ is } \langle T\rangle g \mid \mathit{case}\ e\ \overline{(g(x_g) \Rightarrow e_g)} \mid \bullet \mid \mathbb{B} \mid \mathbb{Z} \\ & \mid \mathsf{One}(e) \mid \mathsf{Map}\langle\tau_k,\tau_v\rangle\{\} \mid e \in e \mid e[e] \mid e[e := e] \mid e = e \mid e < e \mid \neg e \mid e\ \mathsf{op}\ e \mid x\end{aligned}$$

$$\begin{aligned}\text{Stmt} \quad s ::= {} & x = e \mid \langle T\rangle\ \{\overline{g : x_g}\} = x \mid x_m.\mathsf{ins}(x_p, x_v) \mid (x_p, x_v) = x_m.\mathsf{del}(e) \mid (x_p, x_t) = \mathsf{new}\ x_k \\ & \mid \tau\ r \mid s\,;\,s \mid \mathsf{if}\ e\ \mathsf{then}\ s\ \mathsf{else}\ s \mid \mathsf{match}\ x\ \overline{(g(x_g) \Rightarrow s_g)} \\ & \mid \mathsf{async}\ f(\overline{r}) \mid \mathsf{call}\ \overline{q} = f(\overline{r}) \mid \mathsf{assert}\ e \mid \mathsf{skip}\end{aligned}$$

$$\text{Decl} \quad d ::= \mathsf{struct}\ \langle T\rangle\{\overline{g : \tau_g}\} \mid \mathsf{union}\ \langle T\rangle\{\overline{g : \tau_g}\} \mid \mathsf{atomic?}\ \mathsf{fn}\ f(\overline{r : \tau}) \to (\overline{q : \tau})\ \{s\} \mid \mathsf{global}\ a : \tau$$

$$\text{Program} \quad \Sigma ::= \overline{d} \qquad \text{Tokens} \quad l \in \mathbb{T} \qquad \text{Locals} \quad p, q, r \qquad \text{Globals} \quad a, b, c$$

Fig. 1. Syntax of NPL

4 NPL Syntax and Semantics

This section introduces NPL formally. Figure 1 presents its grammar, while Section 4.1 describes its operational semantics. Besides the standard unit, bool, and int, NPL has four primitive types central to its permission system: One, Map, Tag, and Tok. The most important of these is One$\langle\tau\rangle$: a variable of this type stores a single *permission* of type τ, and it is the only kind of value the availability checker tracks linearly. Map$\langle\tau_k,\tau_v\rangle$ denotes a map from keys of type τ_k to values of type τ_v; if either type stores permissions, so does the map. Tok represents a token whose values can be freshly generated, and Tag$\langle\tau\rangle$ associates such a token with a value of type τ. Internally, Tag$\langle\tau\rangle$ is simply stored as a struct containing two fields: `tok:Tok` and `tag:`τ. The type Opt$\langle\tau\rangle$ represents an optional type, with branches `None` and `Some:`τ, that may contain values of type τ. Finally, NPL has standard product and sum types (tagged unions), defined by a set of labels G, which are denoted by named type T in the grammar. Type declarations (discussed later) are used to define (possibly mutually) recursive struct and union types.

Expressions. Most of the expressions are standard, with only a few additions for NPL's primitive types. A struct expression of named type T is constructed using '$\langle T\rangle\{\overline{g : e}\}$' and $e.g$ projects field g back out of a struct. Here, and throughout the grammar, an overline denotes a sequence, one for each field $g \in G$. The struct notation is reused to extract the token or tag out of a Tag$\langle\tau\rangle$ via the field names e.tok and e.tag, respectively. Similarly, a sum-type expression of type T is constructed with $\langle T\rangle g(e)$. Dually, 'e is $\langle T\rangle g$' tests whether a sum-type expression e is currently tagged with label g, and $\mathit{case}\ e\ \overline{(g(x_g) \Rightarrow e_g)}$ is the expression-level counterpart of the **`match`** statement, binding x_g to e's payload when e is tagged g. Unit, boolean, and integer literals are described using $\bullet$, $\mathbb{B}$, and $\mathbb{Z}$ respectively. One(e) constructs an instance of One directly from e. At first, it might seem that this creates unsoundness as it allows a permission be duplicated out of thin air. A subtle point here is that creating permissions itself is not unsound, only assigning them to a variable is! Hence, One(e) may not appear on the right-hand side of an assignment, which is mandated by the availability checker (Section 5.1 formalizes exactly where it is allowed). Map$\langle\tau_k,\tau_v\rangle\{\}$ creates an empty new map, $e \in e$ queries key membership in a map, $e[e]$ reads the value at a key (copying it), and $e[e := e]$ functionally updates a map at a key. The remaining expressions are standard: equality, ordering, negation, and other binary operators, together with a plain variable x.

Statements. Most statements are primitive: they directly update the local or global store. Assignment is expressed as $x = e$, where the availability checker restricts $\mathsf{One} \notin e$ to prevent permission duplication. Its dual, $\langle T \rangle\ \{\overline{g : x_g}\} = x$, is the elimination form for a struct, unpacking x's fields into the variables x_g. Maps allow insertions and deletions: $x_m.\mathsf{ins}(x_p, x_v)$ inserts key x_p and value x_v into map x_m. Deletion, $(x_p, x_v) = x_m.\mathsf{del}(e)$, is subtly different: it takes an expression e identifying the key to remove; the removed key is bound to x_p, while the removed value is bound to x_v. An implicit assert $e \in x_m$ is inserted before every such deletion, to ensure the key being deleted actually exists. Fresh tokens are minted using $(x_p, x_t) = \mathsf{new}\ x_k$, where x_k has type $\mathsf{Set}\langle\tau\rangle$. This statement mints a single fresh token t, binds x_t to $\mathit{One}(t)$, and collects into x_p a permission $\mathit{One}(\langle\mathsf{Tag}\rangle\{\mathsf{tok} : t, \mathsf{tag} : k\})$ pairing each element k in x_k with the new token t. Finally, $\tau\ r$ declares a local variable r of type τ. NPL distinguishes two distinct sets of variables: *local* variables p, q, r, which are declared inside function bodies and are private to the executing function, and *global* variables a, b, c, which are declared at the top level and shared by all threads. We use x, y for variables of either kind, wherever the grammar permits both.

Sequencing is expressed using $s\,;\,s$ while a conditional statement is written as $\mathsf{if}\ e\ \mathsf{then}\ s\ \mathsf{else}\ s$, which branches on the value of e. Similarly, $\mathsf{match}\ x\ \overline{(g(x_g) \Rightarrow s_g)}$ branches on the label of a sum-type variable x, executing s_g if x is tagged with constructor g after binding x_g to the value inside x. An asynchronous call is written as $\mathsf{async}\ f(\overline{r})$ spawning f as a new thread without relinquishing control, while $\mathsf{call}\ \overline{q} = f(\overline{r})$ calls f with arguments $\overline{r}$, waits for it to return, and assigns its return values to $\overline{q}$. Note that asynchronous calls are not allowed to return any values, hence a simple $\mathsf{async}\ f(\overline{r})$ suffices. The grammar requires the arguments and return targets of calls, ordinary and asynchronous, to be local variables. Finally, $\mathsf{assert}\ e$ expresses a boolean conditional that must hold on the program state. Lastly, skip is a no-op.

Declarations. NPL allows three kinds of declarations: types, functions, and variables. Type declarations, struct and union, introduce named product and sum types, respectively, each defined by a name T and a set of fields or labels G. A function declaration $\mathsf{fn}\ f(\overline{r : \tau}) \to (\overline{q : \tau})\ \{s\}$ names its parameters $\overline{r}$ and its return variables $\overline{q}$, both local, allowing a function to return multiple values. Both argument and return variables are passed by value, with the arguments copied into $\overline{r}$ on entry and the final values of $\overline{q}$ copied back to the caller on return. A function may additionally be marked *atomic*, in which case it runs to completion atomically. Finally, a declaration $\mathsf{global}\ a : \tau$ places the global variable a in a global store shared by every thread.

We impose a few requirements on an NPL program not formalized by our type system.

(1) Each atomic function makes only ordinary calls to other atomic functions.
(2) All variable declarations, local and global, use globally unique names.
(3) All local variables of a function, including return variables, are assigned before first use. Return variables must be assigned before function returns since they are implicitly used by the caller.

These requirements are easy to check efficiently using standard static analyses.

A program Σ is simply a sequence of declarations $\overline{d}$. Since every type, function, and global variable is declared at the top level, Σ also serves as the program's *global signature*. Finally, values v are the runtime counterpart of expressions. A value of type $\mathsf{One}\langle\tau\rangle$ has the form $\mathit{One}(v)$, wrapping the permission v it carries, while a map value $\mathit{Map}\{\overline{(v, v)}\}$ is a finite set of key-value pairs. Struct values $\langle T\rangle\{\overline{g : v}\}$ hold a value for every field g, and sum values $\langle T\rangle g(v)$ hold the active label g together with its payload v. The remaining values are tokens l, drawn from a countably infinite universe $\mathbb{T}$, and the unit, boolean, and integer values $\bullet$, $\mathbb{B}$, and $\mathbb{Z}$.

$$\frac{(H,\ell) \vdash e \Downarrow v}{x = e \vdash (H,\ell) \longrightarrow (H,\ell)[x := v]}\ \textsc{Asgn} \qquad \frac{(H,\ell)[x] = \langle T\rangle\{\overline{g : v_g}\}}{\langle T\rangle\{\overline{g : x_g}\} = x \vdash (H,\ell) \longrightarrow (H,\ell)[\overline{x_g := v_g}]}\ \textsc{Unpack}$$

$$\frac{(H,\ell)[x_m] = m \qquad (H,\ell)[x_p] = p \qquad (H,\ell)[x_v] = v}{x_m.\mathsf{ins}(x_p, x_v) \vdash (H,\ell) \longrightarrow (H,\ell)[x_m := m[p := v]]}\ \textsc{MapIns}$$

$$\frac{(H,\ell)[x_m] = m \uplus (p, v) \qquad (H,\ell) \vdash e \Downarrow p}{(x_p, x_v) = x_m.\mathsf{del}(e) \vdash (H,\ell) \longrightarrow (H,\ell)[x_m := m][x_p := p][x_v := v]}\ \textsc{MapDel}$$

$$\frac{(H,\ell)[x_k] = \mathit{Map}\{\overline{(k, \bullet)}\} \qquad l \text{ fresh} \qquad m = \mathit{Map}\{\overline{(\mathit{One}(\langle \mathsf{Tag}\rangle\{\mathsf{tok} : l, \mathsf{tag} : k\}), \bullet)}\}}{(x_p, x_t) = \mathsf{new}\ x_k \vdash (H,\ell) \longrightarrow (H,\ell)[x_p := m][x_t := \mathit{One}(l)]}\ \textsc{New}$$

$$\frac{}{\tau\ r \vdash (H,\ell) \longrightarrow (H,\ell)}\ \textsc{Local} \qquad \frac{(H,\ell) \vdash e \Downarrow \mathsf{true}}{\mathsf{assert}\ e \vdash (H,\ell) \longrightarrow (H,\ell)}\ \textsc{Assert}$$

Fig. 2. Semantics of primitive statements, using judgment $s \vdash (H,\ell) \longrightarrow (H',\ell')$

4.1 Semantics

NPL has a standard call-by-value semantics, expressed using multiset rewriting [9]; Figures 2 and 3 show selected rules. At runtime, a program state is represented by a *configuration* $\langle H, S\rangle$, consisting of (i) a global store H, a finite map from global variables to their values, and (ii) a multiset of threads S, where each thread $T = F^*$ is a stack of frames, one frame per function call. A stack frame is a pair $F = (\ell, s)$, where s is the statement that remains to be executed and ℓ is the frame's local store, a finite map from local variables to their values. Evaluation is defined using a *statement context* [20] which is used to identify the next statement to be executed. A statement context SC is a statement with a "hole" [ ] in place of the next statement to be executed, and SC$[s]$ denotes the operation of filling the hole in SC with statement s.

Most statements (assignment, unpacking, map operations, and local variable declarations) are primitive: they either update the global store H or the local store ℓ of the executing frame (or both). Function calls, in contrast, also change the structure of the configuration itself, by pushing, popping, or spawning frames, and are described separately below.

Figure 2 provides the rules for primitive statements, using the judgment $s \vdash (H,\ell) \longrightarrow (H',\ell')$ which states that executing the primitive statement s, starting from global store H and the local store ℓ of the executing frame, produces the updated stores H' and ℓ'. Expressions are side-effect free and are evaluated by a standard big-step judgment $(H,\ell) \vdash e \Downarrow v$, stating that e evaluates to value v under the two stores. We write $(H,\ell)[x]$ for the value of variable x, looked up in ℓ if x is a local and in H if it is a global, and $(H,\ell)[x := v]$ for the corresponding update. Asgn evaluates e and overwrites x with the result, while Unpack binds each variable x_g to the value of field g of struct x. MapIns looks up x_m to m, x_p to p, and x_v to v, and inserts the key-value pair (p, v) into map m. MapDel evaluates e to the key p to be deleted, removes the entry (p, v) from x_m, and binds x_p and x_v to the removed key and value. Since the logical reasoning engine discharges all asserts before execution, we are guaranteed that key p already exists in x_m. New looks up the set x_k, picks a token l that is fresh, i.e., does not occur anywhere in the current configuration, and binds x_t to the permission $\mathit{One}(l)$ and x_p to the set containing, for each element k of x_k, the permission $\mathit{One}(\langle \mathsf{Tag}\rangle\{\mathsf{tok} : l, \mathsf{tag} : k\})$. Local is a no-op: declaring r does not bind it in the local store. This is

sound because we assume that variables are always assigned before they are used, so r is inserted into the store on its first assignment. Finally, Assert steps only if e evaluates to true; otherwise, the thread is blocked. But again, assuming the logical reasoning engine discharges all assertions, we are guaranteed that a thread is never blocked.

Control Flow and Function Calls. The remaining statements are described in Figure 3 using the judgment $\langle H, S\rangle \longrightarrow \langle H', S'\rangle$, which steps a configuration by executing the next statement of one of its threads. In each step, a thread may be chosen nondeterministically, except when a thread is already executing an atomic function in which case that specific thread must be chosen. To express this constraint, we use the predicate $\mathsf{NonAtomic}(T')$, which holds when the top frame of thread T' does not belong to an atomic function. Checking only the top frame suffices because an atomic function may only call other atomic functions (enforced by the type system, Section 5.1). Every rule has the constraint $\forall T' \in S.\ \mathsf{NonAtomic}(T')$, where S indicates the set of threads excluding the chosen thread.

The statement context SC determines the next executing statement. Rule Step connects the two judgments: if the top frame of some thread is about to execute a primitive statement s, and s updates the stores from (H, ℓ) to (H', ℓ') according to Figure 2, then the configuration steps accordingly, with s replaced by skip. Unlike primitive statements, the remaining statements replace the current statement with another one (using SC), or push, pop, or spawn frames. IfT and IfF evaluate the condition e and continue with s_1 or s_2 in SC, respectively. Match looks up the sum value $\langle T\rangle g(v)$ stored in x, binds its payload v to x_g, and continues with the branch s_g for its active label g.

Call executes call $\overline{q} = f(\overline{r})$ by pushing a new frame (ℓ', s') on top of the caller's frame, where s' is the body of f and ℓ' binds each parameter r' of f to the value of the corresponding argument r in the caller's local store. Since arguments are passed by value, the callee operates on its own copies, and its local store initially contains only its parameters. The caller's frame keeps the call statement in place while the callee executes, so that the return knows where to deliver its results. Ret performs the dual step: once the callee's frame reaches skip, it is popped, the values of the callee's return variables $\overline{q'}$ are copied into the caller's variables $\overline{q}$, yielding ℓ'', and the call statement in the caller's frame is replaced by skip. Return values are thus passed by value as well.

An asynchronous call async $f(\overline{r})$, described by Async, binds the parameters of f exactly as Call does, but instead of pushing the new frame onto the caller's stack, it adds it to the configuration as a new thread consisting of that single frame. The caller does not wait for f: its call statement is immediately replaced by skip and it continues executing. Finally, End removes a thread once its last remaining frame has finished executing. Note that the rules in Figure 3 may apply to any thread in the configuration, so execution interleaves the threads nondeterministically, one statement at a time. The only exception to this rule is that a thread executing an atomic function runs without interference from other atomic functions, as discussed above.

5 Type and Availability Checker

This section defines when an NPL program is well-formed, which requires passing two checks. Type checking (Section 5.1) treats NPL as an ordinary typed language, ensuring that every variable only ever holds values of its declared type. Availability checking (Section 5.2) then enforces the linear treatment of types that carry permissions, guaranteeing that no permission is ever duplicated. Owing to NPL's language design, both checks run in linear time in the size of the program.

5.1 Type Checking

Since NPL's linearity restrictions apply only to types that carry permissions, we separate them from ordinary type checking. Type checking, described in this section, ignores linearity altogether: it

$$\frac{s \vdash (H, \ell) \to (H', \ell')}{\langle H, S \uplus (\ell, \mathsf{SC}[s]) \cdot T\rangle \to \langle H', S \uplus (\ell', \mathsf{SC}[\mathsf{skip}]) \cdot T\rangle}\ \textsc{Step}$$

$$\frac{(H, \ell) \vdash e \Downarrow \mathsf{true}}{\langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{if}\ e\ \mathsf{then}\ s_1\ \mathsf{else}\ s_2]) \cdot T\rangle \to \langle H, S \uplus (\ell, \mathsf{SC}[s_1]) \cdot T\rangle}\ \textsc{IfT}$$

$$\frac{(H, \ell) \vdash e \Downarrow \mathsf{false}}{\langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{if}\ e\ \mathsf{then}\ s_1\ \mathsf{else}\ s_2]) \cdot T\rangle \to \langle H, S \uplus (\ell, \mathsf{SC}[s_2]) \cdot T\rangle}\ \textsc{IfF}$$

$$\frac{(H, \ell)[x] = \langle T\rangle g(v)}{\langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{match}\ x\ (\overline{g(x_g) \Rightarrow s_g})]) \cdot T\rangle \to \langle H, S \uplus (\ell[x_g \mapsto v], \mathsf{SC}[s_g]) \cdot T\rangle}\ \textsc{Match}$$

$$\frac{\mathsf{fn}\ f(\overline{r' : \tau}) \to (\overline{q' : \tau'})\ \{s'\} \in \Sigma \qquad \ell' = [\overline{r' \mapsto \ell(r)}]}{\langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{call}\ \overline{q} = f(\overline{r})]) \cdot T\rangle \to \langle H, S \uplus (\ell', s') \cdot (\ell, \mathsf{SC}[\mathsf{call}\ \overline{q} = f(\overline{r})]) \cdot T\rangle}\ \textsc{Call}$$

$$\frac{\mathsf{fn}\ f(\overline{r' : \tau}) \to (\overline{q' : \tau'})\ \{s'\} \in \Sigma \qquad \ell' = [\overline{r' \mapsto \ell(r)}]}{\langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{async}\ f(\overline{r})]) \cdot T\rangle \to \langle H, S \uplus (\ell, \mathsf{SC}[\mathsf{skip}]) \cdot T \uplus (\ell', s')\rangle}\ \textsc{Async}$$

$$\frac{\mathsf{fn}\ f(\overline{r' : \tau}) \to (\overline{q' : \tau'})\ \{s'\} \in \Sigma \qquad \ell'' = \ell[\overline{q \mapsto \ell'(q')}]}{\langle H, S \uplus (\ell', \mathsf{skip}) \cdot (\ell, \mathsf{SC}[\mathsf{call}\ \overline{q} = f(\overline{r})]) \cdot T\rangle \to \langle H, S \uplus (\ell'', \mathsf{SC}[\mathsf{skip}]) \cdot T\rangle}\ \textsc{Ret}$$

$$\frac{}{\langle H, S \uplus (\ell, \mathsf{skip})\rangle \to \langle H, S\rangle}\ \textsc{End}$$

Fig. 3. Semantics of non-primitive statements, using judgment $\langle H, S\rangle \to \langle H', S'\rangle$

$$\frac{\mathsf{struct}\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \overline{\Gamma \vdash e_g : \tau_g}}{\Gamma \vdash \langle T\rangle\{\overline{g : e_g}\} : T}\ \textsc{Pack} \qquad \frac{\mathsf{struct}\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash e : T}{\Gamma \vdash e.g : \tau_g}\ \textsc{Proj}$$

$$\frac{\Gamma \vdash e : \mathsf{One}\langle\tau\rangle}{\Gamma \vdash e.\mathsf{val} : \tau}\ \textsc{Val} \qquad \frac{\mathsf{union}\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash e : \tau_g}{\Gamma \vdash \langle T\rangle g(e) : T}\ \textsc{Inj} \qquad \frac{\mathsf{union}\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash e : T}{\Gamma \vdash e\ \mathsf{is}\ \langle T\rangle g : \mathsf{bool}}\ \textsc{Is}$$

$$\frac{\mathsf{union}\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash e : T \qquad \overline{\Gamma, x_g : \tau_g \vdash e_g : \tau}}{\Gamma \vdash \mathit{case}\ e\ (\overline{g(x_g) \Rightarrow e_g}) : \tau}\ \textsc{Case} \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{One}(e) : \mathsf{One}\langle\tau\rangle}\ \textsc{One}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{Map}\langle\tau_k, \tau_v\rangle \qquad \Gamma \vdash e_2 : \tau_k}{\Gamma \vdash e_2 \in e_1 : \mathsf{bool}}\ \textsc{Mem} \qquad \frac{\Gamma \vdash e_1 : \mathsf{Map}\langle\tau_k, \tau_v\rangle \qquad \Gamma \vdash e_2 : \tau_k}{\Gamma \vdash e_1[e_2] : \tau_v}\ \textsc{Get}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{Map}\langle\tau_k, \tau_v\rangle \qquad \Gamma \vdash e_2 : \tau_k \qquad \Gamma \vdash e_3 : \tau_v}{\Gamma \vdash e_1[e_2 := e_3] : \mathsf{Map}\langle\tau_k, \tau_v\rangle}\ \textsc{Upd} \qquad \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{Var}$$

Fig. 4. Selected expression typing rules, using judgment $\Gamma \vdash e : \tau$

checks the program as in any typed language, and ensures that variables of type τ only hold values of type τ. Type checking begins with type declarations themselves, which must be well-founded and inhabited, i.e., every declared type must have at least one finite value. Concretely, every union must have at least one variant and struct type must have at least one field, and every cycle of (possibly mutually) recursive declarations must pass through either a union with a non-recursive variant or a Map, whose empty value provides a base case.

The type system uses three judgments, all relative to the *global signature* Σ, which collects all type, function, and global variable declarations, and to a typing context Γ, which assigns types to the variables currently in scope. For expressions, $\Gamma \vdash e : \tau$ states that e has type τ. For statements, $\Gamma \vdash s \dashv \Gamma'$ states that s is well typed in the body of a function, and that executing it extends the variables in scope from Γ to Γ'. Additionally, since Σ is fixed throughout, we omit it from the judgments.

Figure 4 gives selected typing rules for expressions, which are mostly standard. Pack constructs a struct from its fields by checking field e_g has type τ_g for each g, and Proj projects field g back out to type τ_g. Val similarly extracts the value wrapped in a permission of type $\mathsf{One}\langle\tau\rangle$. Inj constructs a value of a union type from $e : \tau_g$ using variant g, and Is tests its active label. Case is the only rule where new variables are introduced into the context Γ. We check e has union type T, and each branch e_g is checked with the payload x_g added to the context, and all branches must have the same type τ. One wraps any value of type τ as a permission of type $\mathsf{One}\langle\tau\rangle$. The map expressions behave as expected: Mem tests whether a key is present, hence e_2 must have e_1's key type; the same holds for Get that reads the value at key e_2 from map e_1; and Upd returns an updated copy of the map requiring e_2 and e_3 to be the key and value type respectively. The remaining rules, for literals, comparisons, negation, arithmetic operators, and variables, are standard, and in particular, Var looks up local variables in Γ.

Figure 5 gives selected typing rules for statements. Rules Asgn through Assert do not depend on whether the enclosing function is atomic, and are therefore written with the mode-agnostic $\Gamma \vdash s \dashv \Gamma'$. Asgn requires the variable x and the expression e to have the same type, while Unpack requires x to have struct type T and each variable x_g to have the type τ_g of the corresponding field. Ins requires the key x_p and value x_v to match the key and value types of the map x_m. Del requires e to have the key type of x_m, and the variables x_p and x_v, which receive the removed key and value, to have the key and value types, respectively. New requires x_k to be a set of values of some type τ (since $\mathsf{Map}\langle\tau, \mathsf{unit}\rangle$ is equivalent to $\mathsf{Set}\langle\tau\rangle$), the token x_t to have type $\mathsf{One}\langle\mathsf{Tok}\rangle$, and x_p to be a set of permissions of type $\mathsf{One}\langle\mathsf{Tag}\langle\tau\rangle\rangle$. Local requires the local r not to be in scope already, and extends the context with $r : \tau$. Seq threads the context through sequential composition. Match requires x to have a union type T and checks each branch s_g with the payload variable x_g, which must not already be in scope, added to the context at the type τ_g of variant g. Since only one branch is executed, any variables a branch introduces, including x_g, go out of scope at the end of the match, which therefore leaves the context unchanged in the conclusion. Assert requires its condition to be a boolean. Async checks that the callee f has no return variables. Both Async and Call check that the type of each actual matches the corresponding formal. The omitted rules, for skip and conditionals, are standard.

5.2 Availability Checking

Availability checking forms the second phase of the type checker, and models permissions and their transfer between variables. It imposes a linear discipline on the subset of NPL's types that carry permissions guaranteeing that a value of such a type may be discarded, but never duplicated. Central to this reasoning is *permission transfer* that occurs in primitive statements: the availability checker checks that the permissions on the lhs of an assignment is a subset of permissions on the

$$\frac{\Gamma \vdash x : \tau \qquad \Gamma \vdash e : \tau}{\Gamma \vdash x = e \dashv \Gamma}\ \textsc{Asgn} \qquad \frac{\text{struct } \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash x : T \qquad \overline{\Gamma \vdash x_g : \tau_g}}{\Gamma \vdash \langle T\rangle\{\overline{g : x_g}\} = x \dashv \Gamma}\ \textsc{Unpack}$$

$$\frac{\Gamma \vdash x_m : \mathsf{Map}\langle \tau_1, \tau_2\rangle \qquad \Gamma \vdash x_p : \tau_1 \qquad \Gamma \vdash x_v : \tau_2}{\Gamma \vdash x_m.\mathsf{ins}(x_p, x_v) \dashv \Gamma}\ \textsc{Ins}$$

$$\frac{\Gamma \vdash x_m : \mathsf{Map}\langle \tau_1, \tau_2\rangle \qquad \Gamma \vdash e : \tau_1 \qquad \Gamma \vdash x_p : \tau_1 \qquad \Gamma \vdash x_v : \tau_2}{\Gamma \vdash (x_p, x_v) = x_m.\mathsf{del}(e) \dashv \Gamma}\ \textsc{Del}$$

$$\frac{\Gamma \vdash x_k : \mathsf{Map}\langle \tau, \mathsf{unit}\rangle \qquad \Gamma \vdash x_t : \mathsf{One}\langle \mathsf{Tok}\rangle \qquad \Gamma \vdash x_p : \mathsf{Map}\langle \mathsf{One}\langle \mathsf{Tag}\langle \tau\rangle\rangle, \mathsf{unit}\rangle}{\Gamma \vdash (x_p, x_t) = \mathsf{new}\ x_k \dashv \Gamma}\ \textsc{New}$$

$$\frac{r \notin \mathrm{dom}(\Gamma)}{\Gamma \vdash \tau\ r \dashv \Gamma, r : \tau}\ \textsc{Local} \qquad \frac{\Gamma \vdash s_1 \dashv \Gamma_1 \qquad \Gamma_1 \vdash s_2 \dashv \Gamma_2}{\Gamma \vdash s_1\ ;\ s_2 \dashv \Gamma_2}\ \textsc{Seq}$$

$$\frac{\text{union } \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \Gamma \vdash x : T \qquad \overline{x_g \notin \mathrm{dom}(\Gamma)} \qquad \overline{\Gamma, x_g : \tau_g \vdash s_g \dashv \Gamma_g}}{\Gamma \vdash \mathsf{match}\ x\ (\overline{g(x_g) \Rightarrow s_g}) \dashv \Gamma}\ \textsc{Match}$$

$$\frac{\Gamma \vdash e : \mathsf{bool}}{\Gamma \vdash \mathsf{assert}\ e \dashv \Gamma}\ \textsc{Assert} \qquad \frac{\text{atomic? fn } f(\overline{r' : \tau}) \in \Sigma \qquad \overline{\Gamma \vdash r : \tau}}{\Gamma \vdash \mathsf{async}\ f(\overline{r}) \dashv \Gamma}\ \textsc{Async}$$

$$\frac{\text{atomic? fn } f(\overline{r' : \tau}) \to (\overline{q' : \tau'}) \in \Sigma \qquad \overline{\Gamma \vdash r : \tau} \qquad \overline{\Gamma \vdash q : \tau'}}{\Gamma \vdash \mathsf{call}\ \overline{q} = f(\overline{r}) \dashv \Gamma}\ \textsc{Call}$$

Fig. 5. Selected statement typing rules using the judgment $\Gamma \vdash s \dashv \Gamma'$.

rhs of an assignment. Ultimately, this will help us prove that the permissions stored in *all available* variables are pairwise disjoint.

Linear and Ordinary Types. The first step in designing the availability checker is to categorize types that may contain permissions. A type is defined to be *linear*, written τ lin, if its values may contain permissions, i.e., if it transitively contains $\mathsf{One}\langle\tau\rangle$, whether directly or inside a map, struct, or union. This is captured by the following inductive definition:

$$\frac{}{\mathsf{One}\langle\tau\rangle\ \mathsf{lin}}\ \textsc{L-One} \qquad \frac{\tau_k\ \mathsf{lin}}{\mathsf{Map}\langle\tau_k, \tau_v\rangle\ \mathsf{lin}}\ \textsc{L-Key} \qquad \frac{\tau_v\ \mathsf{lin}}{\mathsf{Map}\langle\tau_k, \tau_v\rangle\ \mathsf{lin}}\ \textsc{L-Val} \qquad \frac{\tau\ \mathsf{lin}}{\mathsf{Tag}\langle\tau\rangle\ \mathsf{lin}}\ \textsc{L-Tag}$$

$$\frac{\tau\ \mathsf{lin}}{\mathsf{Opt}\langle\tau\rangle\ \mathsf{lin}}\ \textsc{L-Opt} \qquad \frac{(\text{struct} \mid \text{union})\ \langle T\rangle\{\overline{g : \tau_g}\} \in \Sigma \qquad \tau_g\ \mathsf{lin}\ \text{for some } g}{T\ \mathsf{lin}}\ \textsc{L-Named}$$

Since struct and union declarations may be recursive, we take the least relation closed under these rules, so a recursive type that never reaches $\mathsf{One}\langle\tau\rangle$ is not linear. All other types are *ordinary*, written τ ord; in particular, Tok, unit, bool, and int are ordinary. Only variables of linear type are tracked by availability checking: values of ordinary type carry no permissions, and may therefore be copied freely. Since availability checking runs on programs that have already been type checked, we assume knowledge of the type of every variable x, which we write as τ_x.

Permission Transfer. At its core, availability checking tracks how permissions are transferred between variables across an assignment. Whether assignments are explicit (e.g., $x = e$) or implicit (e.g., struct unpack, match, function calls), permission transfer works uniformly, i.e., from source variables into destination variables. We therefore describe each transfer as a sequence of two primitive operations: $\dot{-}x$ *consumes* a source x, requiring it to be available before the assignment, and making it unavailable after; dually, $\dot{+}x$ *provides* a destination x, making it available. Both operations have no effect on variables of ordinary type, which carry no permissions. We formalize this as a judgment $\Delta \bullet P \bullet \Delta'$ stating that performing the sequence of operations P, starting from the set of available variables Δ, results in the set Δ'. The judgment is defined by the following rules:

$$\frac{\tau_x \text{ lin}}{(\Delta \cup \{x\}) \bullet [\dot{-}x] \bullet \Delta}\ \textsc{Lin}^- \qquad \frac{\tau_x \text{ lin}}{\Delta \bullet [\dot{+}x] \bullet (\Delta \cup \{x\})}\ \textsc{Lin}^+ \qquad \frac{\tau_x \text{ ord}}{\Delta \bullet [\dot{-}x] \bullet \Delta}\ \textsc{Ord}^-$$

$$\frac{\tau_x \text{ ord}}{\Delta \bullet [\dot{+}x] \bullet \Delta}\ \textsc{Ord}^+ \qquad \frac{\Delta \bullet [o] \bullet \Delta'' \qquad \Delta'' \bullet [P] \bullet \Delta'}{\Delta \bullet [o \cdot P] \bullet \Delta'}\ \textsc{Flow-Cons} \qquad \frac{}{\Delta \bullet [] \bullet \Delta}\ \textsc{Flow-Nil}$$

Rule $\textsc{Lin}^-$ consumes a linear variable x: it requires x to be available and removes it from the available set. Dually, $\textsc{Lin}^+$ provides a linear variable x by adding it to the available set. Rules $\textsc{Ord}^-$ and $\textsc{Ord}^+$ make both operations no-ops as ordinary variables carry no permissions. Finally, Flow-Cons and Flow-Nil perform a sequence of operations from left to right, threading the set of available variables through it. Order matters within a transfer: all sources are consumed before any destination is provided, so a variable can safely be both a source and a destination of the same transfer (comes in handy for map operations where maps are both sources and destinations).

Availability Rules. Availability checking uses the judgment $\Delta \vdash s \dashv \Delta'$, which states that if the set of available variables before executing s is Δ, then the set of available variables afterwards is Δ'. As in type checking, the signature Σ is fixed, and we omit it.

Figure 6 describes the availability checking rules. LAsgn is the most important rule because it allows permission transfer from an *expression* to a variable via $x = e$, where the destination x is linear. It conservatively treats every free variable of e, denoted by $\text{fvars}(e)$, as a source of permissions, consuming each of them, and then provides x. To prevent construction of permissions out of thin air, e must not contain the expression $\mathit{One}(e')$ anywhere. This is the only place where *One* is restricted; in all other positions, constructing a permission is harmless, because the result is never assigned to a variable. OAsgn applies when x is ordinary and leaves availability unchanged. Local leaves availability unchanged: a newly declared local is therefore not available until it is assigned.

All subsequent rules are simpler as they only transfer permissions across variables. Unpack consumes the struct x and provides each of its fields x_g; since its source is a variable rather than an expression, it needs no separate linear and ordinary cases. The map rules always consume the map they operate on and provide it again afterwards. MapIns additionally consumes the inserted key x_p and value x_v, whose permissions now reside in the map. MapDel instead provides the removed key and value, transferring their permissions out of the map. Note that it does not consume the free variables of the key expression e: the value of e is only used to locate the entry, and does not store any permissions itself. New provides both of its results, the fresh token x_t and the set of tagged permissions x_p, all of which are guaranteed to be fresh. It requires the elements of the set x_k to be of ordinary type τ, making this the only rule that constructs permissions.

Seq threads the available variables through sequential composition. If checks both branches from the same available variables and keeps only the variables available after both of them, and

$$\frac{\tau_x \text{ lin} \qquad \mathit{One} \notin e \qquad \Delta \bullet [\dot{-}y \mid y \in \mathsf{fvars}(e)] \cdot [\dot{+}x] \bullet \Delta'}{\Delta \vdash x = e \dashv \Delta'}\ \textsc{LAsgn} \qquad \frac{\tau_x \text{ ord}}{\Delta \vdash x = e \dashv \Delta}\ \textsc{OAsgn}$$

$$\frac{}{\Delta \vdash \tau\ r \dashv \Delta}\ \textsc{Local} \qquad \frac{\Delta \bullet [\dot{-}x] \cdot [\overline{\dot{+}x_g}] \bullet \Delta'}{\Delta \vdash \langle T \rangle \{\overline{g : x_g}\} = x \dashv \Delta'}\ \textsc{Unpack}$$

$$\frac{\Delta \bullet [\dot{-}x_m,\ \dot{-}x_p,\ \dot{-}x_v,\ \dot{+}x_m] \bullet \Delta'}{\Delta \vdash x_m.\mathsf{ins}(x_p, x_v) \dashv \Delta'}\ \textsc{MapIns} \qquad \frac{\Delta \bullet [\dot{-}x_m,\ \dot{+}x_p,\ \dot{+}x_v,\ \dot{+}x_m] \bullet \Delta'}{\Delta \vdash (x_p, x_v) = x_m.\mathsf{del}(e) \dashv \Delta'}\ \textsc{MapDel}$$

$$\frac{\tau_{x_k} = \mathsf{Map}\langle \tau, \mathsf{unit} \rangle \qquad \tau \text{ ord} \qquad \Delta \bullet [\dot{+}x_p,\ \dot{+}x_t] \bullet \Delta'}{\Delta \vdash (x_p, x_t) = \mathsf{new}\ x_k \dashv \Delta'}\ \textsc{New}$$

$$\frac{\Delta \vdash s_1 \dashv \Delta_1 \qquad \Delta_1 \vdash s_2 \dashv \Delta_2}{\Delta \vdash s_1\ ;\ s_2 \dashv \Delta_2}\ \textsc{Seq} \qquad \frac{\Delta \vdash s_1 \dashv \Delta_1 \qquad \Delta \vdash s_2 \dashv \Delta_2}{\Delta \vdash \mathsf{if}\ e\ \mathsf{then}\ s_1\ \mathsf{else}\ s_2 \dashv \Delta_1 \cap \Delta_2}\ \textsc{If}$$

$$\frac{\overline{\Delta \bullet [\dot{-}x,\ \dot{+}x_g] \bullet \Delta_g} \qquad \overline{\Delta_g \vdash s_g \dashv \Delta'_g}}{\Delta \vdash \mathsf{match}\ x\ (\overline{g(x_g) \Rightarrow s_g}) \dashv \bigcap_g \Delta'_g}\ \textsc{Match} \qquad \frac{\Delta \bullet [\dot{-}r \mid r \in \overline{r}] \bullet \Delta'}{\Delta \vdash \mathsf{async}\ f(\overline{r}) \dashv \Delta'}\ \textsc{Async}$$

$$\frac{\Delta \bullet [\dot{-}r \mid r \in \overline{r}] \bullet \Delta'' \qquad \Delta'' \bullet [\dot{+}q \mid q \in \overline{q}] \bullet \Delta' \qquad \mathcal{G} \subseteq \Delta'' \qquad \mathcal{G} \subseteq \Delta'}{\Delta \vdash \mathsf{call}\ \overline{q} = f(\overline{r}) \dashv \Delta'}\ \textsc{Call}$$

$$\frac{}{\Delta \vdash \mathsf{assert}\ e \dashv \Delta}\ \textsc{Assert} \qquad \frac{\Delta' \subseteq \Delta}{\Delta \vdash \mathsf{skip} \dashv \Delta'}\ \textsc{Skip}$$

Fig. 6. Availability checking rules, using judgment $\Delta \vdash s \dashv \Delta'$

Match does the same for all of its branches, where each branch consumes the matched variable x and provides its payload x_g, to be used for checking branch s_g.

The call rules treat a call as an assignment from its arguments to its return targets. Async consumes the arguments, which are transferred to the newly spawned thread, but provides nothing (as there are no return targets). Call also consumes the arguments $\overline{r}$, whose permissions are transferred to the callee, and then provides the return targets $\overline{q}$, which receive the permissions it returns. In addition, it requires all linear global variables, written $\mathcal{G}$, to be available both when the call is made and when it returns. Since an atomic callee may access any global variable, this lets the callee assume that all globals are available on entry, and requires it to make them available again on exit. This convention, that all globals are available at every function boundary, is what keeps availability checking modular: no interprocedural analysis is needed to determine which globals are available where, so each function body can be checked in isolation, in time linear in its size. Finally, Assert leaves availability unchanged, while Skip may discard any available variables, along with the permissions they hold. This makes the discipline affine rather than strictly linear, allowing a permission that is no longer needed to be discarded at any program point.

5.3 Well-Formed Programs

With both checks in place, we can now define when a program is well formed. A program is *well formed* if all of its type, function, and global variable declarations have distinct names, all of its

$$\frac{\text{atomic fn } f(\overline{r:\tau}) \to (\overline{q:\tau'})\ \{s\} \in \Sigma \qquad \Gamma_\Sigma, \overline{r:\tau}, \overline{q:\tau'} \vdash s \dashv \Gamma' \qquad \Delta_{\text{in}} = \{r \in \overline{r} \mid \tau_r \text{ lin}\} \qquad \Delta_{\text{out}} = \{q \in \overline{q} \mid \tau_q \text{ lin}\} \qquad \mathcal{G} \cup \Delta_{\text{in}} \vdash s \dashv \Delta' \qquad \mathcal{G} \cup \Delta_{\text{out}} \subseteq \Delta'}{\Sigma \vdash f}\ \textsc{FnA}$$

$$\frac{\text{fn } f(\overline{r:\tau}) \to (\overline{q:\tau'})\ \{s\} \in \Sigma \qquad \overline{r:\tau}, \overline{q:\tau'} \vdash s \dashv \Gamma' \qquad \Delta_{\text{in}} = \{r \in \overline{r} \mid \tau_r \text{ lin}\} \qquad \Delta_{\text{out}} = \{q \in \overline{q} \mid \tau_q \text{ lin}\} \qquad \mathcal{G} \cup \Delta_{\text{in}} \vdash s \dashv \Delta' \qquad \mathcal{G} \cup \Delta_{\text{out}} \subseteq \Delta'}{\Sigma \vdash f}\ \textsc{FnNA}$$

Fig. 7. Well-formedness of a function declaration, where $\Gamma_\Sigma = \{a : \tau \mid \text{global } a : \tau \in \Sigma\}$, and $\mathcal{G} = \{a \mid \text{global } a : \tau \in \Sigma \wedge \tau \text{ lin}\}$

type declarations are well-founded and inhabited (Section 5.1), and each of its functions is well formed according to the rules in Figure 7, combining type and availability checking.

Rule FnA checks an atomic function f, combining both checks on its body s. The initial context contains the global variables, as Γ_Σ, together with the parameters $\overline{r}$ and the return variables $\overline{q}$. This allows the function body to access globals and parameters, and to assign to its return variables. For availability checking, and following the convention of rule Call (Section 5.2), the body may assume that all linear globals $\mathcal{G}$ are available on entry, together with its linear parameters Δ_{in}, whose permissions are transferred in by the caller. In turn, the body must make all linear globals available again on exit, together with its linear return variables Δ_{out}, whose permissions are transferred back to the caller. Any other variable still available on exit, such as a linear local, is simply discarded.

Rule FnNA checks a non-atomic function in the same way, with one difference: the initial context contains only the parameters and the return variables, but not the globals, so that the body cannot access global variables at all. Availability checking is the same as for atomic functions: the body must preserve the availability of all linear globals, which holds trivially since it cannot access them, but ensures that every call it makes to an atomic function finds all globals available, as rule Call requires. Since each rule checks a single function body, independently of all others, checking that a program is well formed takes time linear in the size of the program. Passing this check for all defined functions yields a well-formed program.

6 Soundness

This section formalizes NPL's central guarantee, *global disjointness*: in every reachable configuration of a well-formed program, the permissions held by available variables are pairwise disjoint.

Permission Collection. Since disjointness is a statement about the permissions stored in a configuration, we first make precise which permissions a value stores. The function $\text{Perms} : \text{Value} \to \text{Multiset}(\text{Value})$ collects them, following the value syntax of Figure 1:

$$\text{Perms}(\mathit{One}(v)) = \{v\} \qquad \text{Perms}(\mathit{Map}\{\overline{(v_k, v_v)}\}) = \bigcup_{(v_k, v_v)} \big(\text{Perms}(v_k) \cup \text{Perms}(v_v)\big)$$

$$\text{Perms}(\langle T\rangle\{\overline{g : v_g}\}) = \bigcup_{g} \text{Perms}(v_g) \qquad \text{Perms}(\langle T\rangle g(v)) = \text{Perms}(v)$$

$$\text{Perms}(v) = \emptyset \quad \text{if } v \text{ is a token } l, \bullet, \text{ a boolean, or an integer}$$

A value $\mathit{One}(v)$ of type $\text{One}\langle\tau\rangle$ stores exactly one permission, namely v itself. A map stores the permissions of all its keys and values, a struct those of all its fields, and a sum value those of its active variant. Here, and throughout this section, $\cup$ and $\bigcup$ denote multiset union, which adds

multiplicities: a permission stored in two places is counted twice. Tokens, unit, booleans, and integers store no permissions.

To lift Perms to configurations, we need to know the type and availability information of variables at runtime. We achieve this by introducing a notion of a *typed configuration* $\langle \Delta, H, TS \rangle$. This extends an untyped configuration $\langle H, S \rangle$ with the set Δ of available global variables, and replaces each thread by a *typed thread*: a stack of *typed frames* $(f : \Gamma_f; \Delta_f, \ell, s)$, each of which additionally records the name of the function f that the frame executes, the typing context Γ_f assigning types to the frame's local variables in scope, and the set Δ_f of its available local variables. The typing context is what allows us to type check the frame statement s, and the availability context determine which of these variables are linear, hence must count towards stored permissions. The permissions of a typed configuration are those stored in its available variables, both global and local:

$$\mathsf{Perms}\langle \Delta, H, TS \rangle \;=\; \bigcup_{a \in \Delta} \mathsf{Perms}(H(a)) \;\cup\; \bigcup_{T \in TS} \; \bigcup_{(f:\Gamma_f;\Delta_f,\ell,s) \in T} \; \bigcup_{r \in \Delta_f} \mathsf{Perms}(\ell(r))$$

Finally, we write $\mathit{Proj}(TS)$ to define a projection function that recovers the untyped multiset of threads S by erasing the additional type and availability information from every frame of TS.

Disjointness Theorem. A well-typed configuration is defined by the following rule:

$$\frac{\Vdash H \qquad \Delta \Vdash TS \qquad \mathsf{Perms}\langle \Delta, H, TS \rangle \text{ disjoint}}{\Sigma \Vdash \langle \Delta, H, TS \rangle} \textsc{Config}$$

The judgment $\Vdash H$ states that the global store is well typed: for every global $a : \tau$ declared in Σ, $H(a)$ is a value of type τ. The judgment $\Delta \Vdash TS$ states that every frame of every thread of TS is well typed, given that the globals in Δ are available. A typed frame is well typed if it is consistent with the function it executes, as captured by the following rule for the running (top) frame of a thread, where $m = a$ if f is atomic and $m = na$ otherwise, and $\mathcal{G}$ and Δ_{out} are as in Figure 7:

$$\frac{\begin{array}{c}\text{atomic? } \mathsf{fn}\ f(\overline{r : \tau}) \to (\overline{q : \tau'})\ \{s_0\} \in \Sigma \qquad \forall r \in \mathsf{dom}(\Gamma_f).\ \vdash \ell(r) : \Gamma_f(r) \\ \Gamma_f \vdash_m s \dashv_m \Gamma' \qquad \Delta \cup \Delta_f \vdash s \dashv \Delta' \qquad \mathcal{G} \cup \Delta_{\text{out}} \subseteq \Delta'\end{array}}{\Delta \Vdash (f : \Gamma_f; \Delta_f, \ell, s)} \textsc{F-Run}$$

First, the local store must agree with the typing context, assigning to each local r in Γ_f a value of type $\Gamma_f(r)$, where $\vdash v : \tau$ is the standard typing judgment for values. Second, the remaining statement s must pass both checks of Section 5, as if it were the rest of f's body: it must be well typed in Γ_f under the mode of f, and checking its availability, starting from the available globals Δ and locals Δ_f, must meet the exit condition of f, i.e., all linear globals and linear return variables of f must be available at the end. The rule extends naturally to the remaining frames of a thread, and a multiset of threads is then well typed if all frames of each of its threads are well-typed. Finally, $\mathsf{Perms}\langle \Delta, H, TS \rangle$ disjoint states that every permission occurs at most once in the multiset of permissions of the configuration, i.e., the multiset is in fact a set. A well-typed configuration therefore never stores the same permission in two available variables. The following theorem states that every step of execution preserves well-typedness, and hence disjointness.

Theorem 1 (Preservation). *Suppose Σ is well formed. If $\Sigma \Vdash \langle \Delta, H, TS \rangle$ and $\langle H, \mathit{Proj}(TS) \rangle \to \langle H', S' \rangle$, then there exist Δ' and TS' such that $\Sigma \Vdash \langle \Delta', H', TS' \rangle$ and $\mathit{Proj}(TS') = S'$.*

In other words, whenever the projection of a well-typed configuration takes a step, the resulting configuration is also a projection of a well-typed configuration. And since disjointness of permissions is part of well-typedness, global disjointness follows as a consequence of preservation.

NPL also features a standard progress theorem, stating that a runtime configuration can always take a step. The only reason for a configuration to get stuck is an assertion failure. Although NPL's type and availability checks cannot rule out failing assertions, the logical reasoning engine can: verification (Section 6.1) statically discharges every assertion, guaranteeing that it holds at runtime. Under this guarantee, NPL also enjoys progress:

Theorem 2 (Progress). *Suppose* Σ *is well formed,* $\Sigma \Vdash \langle \Delta, H, TS \rangle$*, and every assertion executed by the program holds at runtime. Then either* $Proj(TS)$ *is empty, i.e., every thread has terminated, or there exist* H' *and* S' *such that* $\langle H, Proj(TS) \rangle \longrightarrow \langle H', S' \rangle$*.*

These two theorems exhibit an elegant synergy between NPL's type system and its logical reasoning engine, in which each supplies exactly what the other needs. Preservation holds for every well-formed program, whether or not it has been verified, and guarantees global disjointness in every reachable configuration. Verification (Section 6.1) exploits this guarantee: it injects disjointness as an assumption into the verification conditions, which is what allows proving correctness of the assertions. Once verification establishes that all assertions hold, progress applies, guaranteeing that the program never gets stuck. Thus, the type system supplies disjointness to the logic, and the logic in turn supplies the assertions that progress needs, together yielding the full meta-theoretical guarantees of NPL.

6.1 Verification

The standard methodology for verifying sequential programs is to convert a program annotated with preconditions, postconditions, and loop invariants into a collection of Floyd-Hoare triples. These triples are then translated to first-order verification conditions using *weakest preconditions* and discharged automatically by constraint solvers. NPL follows this general paradigm but also uses yield invariants to constrain the interference between threads. A non-atomic function may call a yield invariant to model yielding control to other threads, so that a concurrent execution is modeled as a sequence of *yield-to-yield* execution fragments, each executed indivisibly by a single thread. The called yield invariant must be established at the call site and preserved by every yield-to-yield fragment of the program. The proof obligations for establishment and preservation of yield invariants are also reducible to Floyd-Hoare triples. Thus, verification of an NPL program ultimately rests on the same foundation as the verification of sequential programs.

NPL is carefully designed such that its expressions are closed under weakest precondition over its statements. If all annotations are written in the NPL expression language, then every verification condition is also an NPL expression. It therefore suffices to translate NPL expressions into first-order logic accepted by SMT solvers such as Z3 and CVC5. The main challenge in this translation is the encoding of partial maps. First-order logic models mutable memory using the select-update theory of total arrays [16]. We encode partial maps natively using total arrays and algebraic datatypes, both available in SMTLIB, avoiding any separate and potentially expensive axiomatization. The key insight behind our encoding is that the type $\mathsf{Map}\langle K, V \rangle$ has a dual interpretation: the operational semantics treats it as a partial map from K to V, while verification may treat it as a total map from K to $\mathsf{Opt}\langle V \rangle$. All operations on maps are equally amenable to either interpretation. For example, `m.ins(a, None())` removes the entry for key `a` from the partial map `m`, and updates the same entry to `None()` in the total map `m`.

7 Experience

We have implemented NPL in the Civl [44] verifier for concurrent programs. Civl allows a concurrent program to be represented at multiple layers of abstraction [43, 45]. These layers are totally-ordered with the lowest layer being the most concrete and the highest layer being the most abstract.

Table 1. Civl results on a 2.00 GHz Intel Core i5-1038NG7 with 4 physical cores and 16 GB DRAM.

Example	Time (s)	Verification Conditions	Code lines	Annotation lines	Proof overhead
GC	25.53	733	461	894	1.94
verified-ft	5.80	312	296	383	1.29
Paxos	6.91	95	175	301	1.72
cache-coherence	9.36	117	217	223	1.03
ABD	3.31	54	97	202	2.08
shared-vector	5.58	77	113	89	0.79
multiset	2.10	77	100	107	1.07
treiber-stack	5.37	17	75	95	1.27
BroadcastConsensus	1.76	21	67	76	1.13
msq	5.71	18	88	39	0.44
StoreBuffer	1.85	21	70	61	0.87
barrier-sync	1.73	10	77	43	0.56
2pc	1.63	8	57	56	0.98
alloc-mem	2.12	19	45	59	1.31
DeviceCache	1.68	31	80	30	0.38
bluetooth	1.87	31	57	54	0.95
seqlock	1.74	43	60	48	0.80
lc-list	5.59	49	75	22	0.29
lease	1.45	14	34	68	2.00
civl-paper	1.93	37	57	41	0.72
MutexOverFutex	1.74	4	57	34	0.60
two-queues	2.16	6	52	20	0.38
leader	1.40	5	37	45	1.22
rc-dyn	1.93	10	54	17	0.31
prefix-scan	1.50	7	48	32	0.67
array-insert	1.77	26	49	25	0.51
PingPong	1.51	10	50	22	0.44
ProducerConsumer	1.40	6	47	21	0.45
reserve	1.59	5	37	34	0.92
2agree	1.59	21	36	33	0.92
alloc-tid	1.47	12	30	38	1.27
tds	1.97	33	42	22	0.52
ticket	1.52	13	30	34	1.13
lock-incr	1.93	26	31	30	0.97
rwlock	1.85	43	43	17	0.40
ogcounter-lock	1.67	27	22	34	1.55
spin-lock	1.62	20	31	26	0.84
write-barrier	1.45	18	35	21	0.60
rc	1.45	8	37	11	0.30
peterson	1.43	2	34	17	0.50
purity	1.52	19	34	16	0.47
parallel-reduction	1.40	3	21	11	0.52
cuda-add	1.45	6	16	4	0.25
ogcounter	1.35	5	10	10	1.00

Transformations from lower to higher layers involve replacing fine-grained procedures with coarse-grained atomic actions as well as changing the program state representation. Civl verifies the

Table 2. Proof overhead comparison between Civl and Raven.

Example	Civl proof overhead	Raven proof overhead
treiber-stack	1.27	1.90
msq	0.44	1.23
lc-list	0.29	0.62
rc-dyn	0.31	2.83
ticket	1.13	1.81
spin-lock	0.84	1.40
peterson	0.50	1.75

connection between adjacent layers, ensuring that the correctness of the highest layer validates the correctness of the lowest.

The Civl open source repository contains 56 benchmarks out of which 44 use permissions. These benchmarks include examples from a variety of application domains: lock implementations, concurrent data structures, distributed protocols, and parallel programs. Table 1 shows the result of running the Civl verifier on these 44 benchmarks. The table contains columns for running time, number of verification conditions, number of code lines, number of annotation lines, and the proof overhead obtained by dividing the number of annotation lines with the number of proof lines. These results show that verification is fast, only a few seconds even for the largest examples, suggesting that interactive code development is feasible. Although not shown in the table, we note that time spent in type checking is negligible compared to the time spent in solving verification conditions.

We offer a comparison of our verifier against Raven, a verifier that similarly targets constraint solvers but is based on concurrent separation logic. The running times reported in our table are similar to those reported by Raven (a few seconds at most). Table 2 compares the proof overhead of Civl and Raven on the collection of common benchmarks. This table shows that the proof overhead of our approach is comparable and often better than that of Raven.

In the rest of this section, we provide a qualitative assesment of the verified programming experience enabled by NPL by presenting a collection of case studies from different domains. It would be tedious and verbose to implement these examples in Civl without the use of permissions.

7.1 Reference counting

This example models *shared mutable memory* where multiple threads simultaneously need live references to the same object. *Reference counting* is the canonical pattern for managing such state safely: each object tracks the number of live references and is deallocated only when that count reaches zero.

```
struct RcData { data: T, all_tags: Set<Tag<int>> }
global rcs: Map<One<Tok>, RcData>
```

The type `RcData` contains fields `data` and `all_tags`. We refer to an instance of `RcData` as a *cell*. The field `all_tags` contains the set of integer tags associated with this cell, where each tag intuitively models a unique live reference. `rcs` stores a pool of shared cells, each accessible by a globally unique token, as expressed by the key type `One<Tok>`.

To prove correctness, NPL associates a *yield invariant* `YieldInv` with each thread's tag; the thread must maintain this invariant whenever it yields control.

$$\textbf{YieldInv}(t : \mathsf{One}\langle\mathsf{Tag}\langle\mathsf{int}\rangle\rangle) \triangleq \mathsf{One}(t.\mathsf{val.tok}) \in \mathsf{rcs} \ \wedge \ t.\mathsf{val} \in \mathsf{rcs}[t.\mathsf{val.tok}].\mathsf{all_tags}$$

This invariant asserts that the cell identified by `t`'s embedded token, `t.val.tok`, exists in `rcs` and that `t.val` belongs to the cell's `all_tags` set. Any thread holding a tag therefore knows the cell remains live despite concurrent execution of other threads. Therefore, a read or write operation may be performed safely while holding a tag.

A thread drops its reference by calling `free`, which consumes the linear tag (linear variables marked in red):

```
atomic fn free(tag: One<Tag<int>>) {
  (one_tok, rc) = rcs.del(One(tag.val.tok)); // remove rc temporarily
  RcData { data, all_tags } = rc;                 // unpack rc
  all_tags.del(tag);                              // consume this reference
  if all_tags = empty then
    return                                        // last ref: one_tok is dropped
  else
    rc = RcData { data, all_tags };               // pack rc
    rcs.ins(one_tok, rc) }                      // reinsert with updated count
```

Because `tag` is a function argument, the caller loses ownership of it after the call. Inside the function body, the cell `rc` is temporarily removed from `rcs` and the thread's tag is deleted from `all_tags`. Then, if `all_tags` is empty, it means that the current thread is the last surviving thread holding a reference to the shared cell. In this case, `one_tok` is simply dropped, meaning the cell can be safely deallocated without compromising memory safety. Otherwise, the cell `rc` is reconstructed and reinserted into `rcs`.

The type system and logical reasoning unite to prove noninterference of `YieldInv` with respect to `free`. Because every `One<Tag<int>>` across all threads is globally unique, no two threads hold the same tag, and as long as a tag is owned by one thread, it cannot be owned by another. When `free` observes that `all_tags` is empty after removing `tag`, it is guaranteed that no other thread holds a reference to this cell, making deallocation safe. The invariant **YieldInv** ensures that `all_tags` contains the tags of all threads holding a reference to the cell, preventing deallocation while any such reference remains.

7.2 Paxos

The Paxos consensus protocol proceeds in increasingly-numbered rounds each owned by a proposer with the goal of getting a collection of acceptors to agree on a common value. The proposer of a round r first asks all acceptors to join that round. An acceptor joins round r if it has previously joined only rounds lower than r. If a quorum of acceptors join round r, the proposer of r proposes a value v to all acceptors asking them to vote for v. An acceptor votes for round r if it has not joined any round greater than r. If a quorum of acceptors vote for round r, then there is guaranteed to be majority consensus on v.

The verification technique of inductive sequentialization [42] recommends that commutativity arguments be used to align an arbitrary execution of Paxos into a simple execution in which rounds execute in increasing order and operations within each round also execute in a fixed total order. In this per-round total order, all Join actions precede all Vote actions and the Join and Vote actions themselves occur in a total order dictated by the acceptor ids. This approach leads to particularly simple invariants that need to hold only for these simple executions. But proving that Paxos actions commute with each other is difficult.

In our new proof of Paxos, we exploit permissions to prove *contextual* commutativity which allows us to apply the inductive sequentialization proof rule easily. It turns out that context-independent commutativity is difficult because, for example, the Vote action by an acceptor for round r commutes earlier in time to a concurrent Join action only if this Join action is for a round larger than r. Consider a partially-sequentialized execution in which the next action to be added to the end is a Vote action for round r. Intuitively, at this time the only Join actions left are for rounds greater than r. If we can capture this fact and add it as an assumption to the commutativity check, we will succeed in proving contextual commutativity.

Permissions allow us to compactly achieve our goal of contextual commutativity checking. We associate a permission with each event handler executed during each round. All permissions needed for a round are handed to the action that starts that round. Whenever the proposer sends a message to an acceptor to either join or vote for a round, it sends the appropriate permission as well along with the message. As an example, here is the atomic action for the Join operation at the at the Civl layer where inductive sequentialization is applied.

```
atomic fn A_Join(r: Round, n: Node, p: One Permission)
asserts Round(r) && p⟶val == JoinPerm(r, n);
{ ... }
```

The `asserts` specification of `A_Join` is required to be valid upon any invocation of `A_Join`. This specification indicates that `r` is a valid round and the permission `p` is the appropriate permission for round `r` and acceptor Node `n`. The presence of these permissions in the input parameters of event handlers allow us to generate the contextual assumption from the structure of the inductive sequentialization invariants. Consequently, the new proof is much more compact compared to the earlier proof.

7.3 Parallel pre-scan

A pre-scan (also known as exclusive prefix sum) is a parallel algorithm that takes as input a sequence of values $\vec{x}$ and computes another sequence $\vec{y}$, where y_0 is 0 and each element y_i for $i > 0$ is the cumulative sum of all elements before index i. The algorithm operates in two phases, upsweep and downsweep, that execute sequentially. The computation in each phase is organized as a tree that is processed level by level. While the computation at each level is performed by several threads in parallel, the levels themselves execute sequentially. The number of threads executing at a level is the same as the number of tree nodes at that level. Therefore, the number of threads running in parallel depends on the level being executed.

We prove data-race freedom and memory safety for this algorithm. The input is an array of integers. The program is written using a orchestrator that divides ownership of array segments among parallel threads at the beginning of each level. When a thread finishes execution at a level, it hands ownership of its array segment back to the orchestrator. Once all threads executing a level finish, the orchestrator repartitions the array among the threads that will execute the next level.

We use a linear map to represent the input array. The top-level function looks as follows:

```
fn PrefixPreScan(A: Map<One<int>,int>, N: int) ⟶ B: Map<One<int>,int> {
  B = UpSweep(A, N);
  ...
  B = DownSweep(B, N);
}
```

`A` is passed into `UpSweep`, inside which it is repeatedly split, updated in parallel, and recombined as the computation goes from one level to the next. There are no global variables; all communication

is done by explicit handoff of linear maps between parent and children threads. The modified map is returned back to `PrefixPreScan` and stored in its local variable `B`, which is similarly processed further by `DownSweep` and finally returned to the caller of `PrefixPreScan`.

The proof is able to take advantage of commutativity reasoning (via movers) implemented in Civl. Since all memory accesses are to local linear maps and hence both movers, the parallel execution throughout the program is reduced to sequential execution, enabling logical reasoning to pretend as if memory safety is being proved on a sequential program.

8 Related Work

There is a vast body of research on reasoning about concurrent programs by modeling them as transition systems [18, 19, 53, 64, 67, 73, 76]. These approaches share our motivation, but are disconnected from our technical approach of placing type and logical annotations directly on program syntax. We therefore focus on approaches based on types and program logics, and organize them by where permissions are tracked. Section 8.1 discusses approaches that track permissions, capabilities, or ownership in the *type system*, beginning with those that, like NPL, use types to simplify verification. Section 8.2 discusses approaches that track permissions in the *logic*, such as concurrent separation logics and permission-based verifiers.

8.1 Permissions in the Type System

The works closest to NPL use a linear or ownership type system specifically to simplify verification. Linear maps [46] initiated the idea of modeling memory as a collection of partial maps with disjoint domains. NPL builds on this idea, but generalizes this to a full-blown programming model with concurrent programming and adds *nesting* of linear maps inside structs and other maps, which is essential for practical data structures. The Civl verifier [30, 43, 45] also uses linear permissions to supply disjointness facts for refinement proofs of concurrent programs but, unlike NPL, this not conducted via the type system. Programmers must explicitly implement the permission collection function on data structures and specify disjointness constraints, both of which are automated by NPL's availability checker. Linear Dafny [51] integrates linear types into Dafny [49] so that linear data can be reasoned about directly, removing the framing obligations that dynamic frames otherwise impose on verification conditions. IronSync [29] builds on Linear Dafny to verify concurrent systems, using owned *ghost* shards of a separately defined localized transition system. However, linear types in Linear Dafny are best suited to tree-shaped data: DAGs and cyclic structures, such as doubly-linked lists, fall back to Dafny's non-linear heap and dynamic frames, and linear and non-linear data are combined using regions [22, 72, 78]. NPL instead represents pointer-based structures directly, distinguishing type Tok from One⟨Tok⟩, so ordinary tokens can be used to model memory locations that are shared and not exclusively owned. Verus [47, 48] similarly verifies Rust programs with an SMT solver [15], using Rust's type system together with linear ghost permissions to track resources. Several other Rust verifiers, including Prusti [3], RustHorn [52], Creusot [17], and Aeneas [32], exploit ownership to eliminate heap reasoning, and Cogent [1] uses linear types to generate code together with a proof of its correctness. The main difference is that these systems rely on *references*, with mutable and immutable borrows, and rely on their type systems for memory safety. NPL instead models references using named linear maps recovering framing syntactically. This makes programs more explicit, but yields a considerably simpler type system with linear-time checking, since NPL's logical reasoning only needs disjointness from the type system, not memory safety. Moreover, NPL uses the same permissions both to model memory and to coordinate concurrent threads, as in the barrier example of Section 3.

Beyond verification, there is a rich history of type systems based on linear logic [24, 25, 74] for a variety of applications, from checking communication protocols (à la session types [8, 75])

to resource management. Closest to NPL are type systems that separate the *name* of a resource from a linear *capability* to access it. The calculus of capabilities [14] tracks linear capabilities for memory regions, and alias types [70, 77] separate freely copyable pointers $ptr(\ell)$ from linear capabilities $\{\ell \mapsto \tau\}$ that grant access to location ℓ, for compiler-controlled memory management in low-level languages. L^3 [54] and linear regions [22] use the same split to support strong updates, which change the type of a memory cell, and region-based memory management. Asynchronous liquid separation types [41] guarantee the absence of races and preserve program invariants in a functional language by combining liquid types [66] and concurrent separation logic. Types are indexed by resource sets and track the effect of program execution on individual heap locations. As a result, types capture ownership of shared state across concurrent tasks and allow reasoning about ownership transfer between tasks. Mezzo [4, 63] makes permissions themselves the types of a programming language, distinguishing duplicable permissions, for immutable data, from exclusive ones, for mutable data. Unlike NPL where sharing mutable data is pervasive, by default, Mezzo does not allow sharing of mutable data but offers a workaround using dynamic checks. NPL shares the separation of names and capabilities with this line of work, but differs in three respects. First, its permissions are abstract: they represent not only memory locations, but equally threads and protocol roles (Section 3). Second, permissions nest through linear maps keyed by permissions, without the need for existentially quantified locations or focusing. Third, its availability checker is deliberately minimal, guaranteeing only disjointness, in linear time, while all other reasoning is delegated to logic.

8.2 Permissions in the Logic

Automated, SMT-based verifiers have long supported permission-based reasoning about concurrent programs, starting with VCC [13], Chalice [50], VeriFast [35, 36], and VerCors [2, 5]. Implicit dynamic frames [69] combine accessibility predicates, which grant permission to access a heap location, with first-order assertions; they underlie Chalice and the Viper infrastructure [56], where fractional and abstract read permissions [31] allow threads to share read access to the same state. More recently, Raven [28] implements concurrent separation logic with fractional permissions on top of first-order constraint solvers. In all of these approaches, permissions are *logical* assertions: the verifier must track which permissions are held at each program point alongside the program state itself, and encode them in verification conditions, typically using quantifiers over the heap. The reliance on first-order logic and SMT solvers is similar to NPL, but NPL tracks permissions in the type system instead. Since the type system already guarantees disjointness, NPL's verification conditions need no permission model and no quantifiers to frame the heap.

Concurrent separation logic (CSL) [6, 7, 61] extends separation logic to concurrent programs: threads own disjoint portions of the heap, and ownership is transferred between threads through resource invariants. Modern CSLs generalize permissions further to user-defined resources. Iris [39, 40, 55, 71] is a higher-order CSL, mechanized in Rocq, in which ghost state built from partial commutative monoids expresses custom protocols between threads. It has been used to prove the soundness of Rust's type system [38], and to verify distributed protocols [68], CRDTs [59], network components [27], distributed databases [26], journaling systems [11], and crash-safe systems [10]. NPL's abstract permissions, which represent threads and protocol roles as well as memory, are similar in spirit to ghost resources in Iris, but are tracked by the type system rather than in the logic. In CSL, framing and functional correctness are both established in the logic, and proofs are largely interactive. As a result, proofs tend to be long: GoJournal [11], for example, a concurrent journaling system with 1,400 lines of code, required 29,000 lines of proof [12]. NPL instead aims for *ergonomic* verification by separating these concerns: a lightweight, linear-time availability checker establishes disjointness, which an automated, first-order logic then assumes.

9 Conclusion

This paper presented NPL, a programming model that separates framing from functional correctness by tracking disjointness in the type system rather than in the logic. NPL represents state as nested linear maps keyed by abstract permissions, which model memory locations, threads, and protocol roles alike, and a linear-time availability checker guarantees that permissions are never duplicated. We proved that well-formed programs preserve a global disjointness invariant under fine-grained concurrency, which classical, first-order verification conditions can then simply assume. We implemented NPL in the Civl verifier and used it to verify 44 benchmarks spanning lock implementations, concurrent data structures, distributed protocols such as Paxos, and parallel programs, with proof overhead comparable to or lower than that of Raven.

References

[1] Sidney Amani, Alex Hixon, Zilin Chen, Christine Rizkallah, Peter Chubb, Liam O'Connor, Joel Beeren, Yutaka Nagashima, Japheth Lim, Thomas Sewell, Joseph Tuong, Gabriele Keller, Toby Murray, Gerwin Klein, and Gernot Heiser. 2016. Cogent: Verifying High-Assurance File System Implementations. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems* (Atlanta, Georgia, USA) *(ASPLOS '16)*. Association for Computing Machinery, New York, NY, USA, 175–188. https://doi.org/10.1145/2872362.2872404

[2] Afshin Amighi, Stefan Blom, Saeed Darabi, Marieke Huisman, Wojciech Mostowski, and Marina Zaharieva-Stojanovski. 2014. *Verification of Concurrent Systems with VerCors*. Springer International Publishing, Cham, 172–216. https://doi.org/10.1007/978-3-319-07317-0_5

[3] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging rust types for modular specification and verification. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 147 (Oct. 2019), 30 pages. https://doi.org/10.1145/3360573

[4] Thibaut Balabonski, François Pottier, and Jonathan Protzenko. 2016. The Design and Formalization of Mezzo, a Permission-Based Programming Language. *ACM Trans. Program. Lang. Syst.* 38, 4, Article 14 (Aug. 2016), 94 pages. https://doi.org/10.1145/2837022

[5] Stefan Blom and Marieke Huisman. 2014. The VerCors Tool for Verification of Concurrent Programs. In *FM 2014: Formal Methods*, Cliff Jones, Pekka Pihlajasaari, and Jun Sun (Eds.). Springer International Publishing, Cham, 127–131.

[6] Stephen Brookes. 2006. Variables as Resource for Shared-Memory Programs: Semantics and Soundness. *Electron. Notes Theor. Comput. Sci.* 158 (May 2006), 123–150. https://doi.org/10.1016/j.entcs.2006.04.008

[7] Stephen Brookes and Peter W. O'Hearn. 2016. Concurrent separation logic. *ACM SIGLOG News* 3, 3 (Aug. 2016), 47–65. https://doi.org/10.1145/2984450.2984457

[8] Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR 2010 - Concurrency Theory*, Paul Gastin and François Laroussinie (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 222–236.

[9] Iliano Cervesato and Edmund S. L. Lam. 2015. Modular Multiset Rewriting. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Martin Davis, Ansgar Fehnker, Annabelle McIver, and Andrei Voronkov (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 515–531.

[10] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2019. Verifying concurrent, crash-safe systems with Perennial. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) *(SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 243–258. https://doi.org/10.1145/3341301.3359632

[11] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. 2021. GoJournal: a verified, concurrent, crash-safe journaling system. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 423–439. https://www.usenix.org/conference/osdi21/presentation/chajed

[12] Tej Chajed, Joseph Tassarotti, Mark Theng, M. Frans Kaashoek, and Nickolai Zeldovich. 2022. Verifying the DaisyNFS concurrent and crash-safe file system with sequential reasoning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 447–463. https://www.usenix.org/conference/osdi22/presentation/chajed

[13] Ernie Cohen, Markus Dahlweid, Mark Hillebrand, Dirk Leinenbach, Michał Moskal, Thomas Santen, Wolfram Schulte, and Stephan Tobies. 2009. VCC: A Practical System for Verifying Concurrent C. In *Theorem Proving in Higher Order Logics (Lecture Notes in Computer Science)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 23–42. https://doi.org/10.1007/978-3-642-03359-9_2

[14] Karl Crary, David Walker, and Greg Morrisett. 1999. Typed memory management in a calculus of capabilities. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Antonio, Texas, USA) *(POPL '99)*. Association for Computing Machinery, New York, NY, USA, 262–275. https://doi.org/10.1145/292540.292564

[15] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.

[16] Leonardo De Moura and Nikolaj Bjørner. 2009. Generalized, efficient array decision procedures. In *2009 Formal Methods in Computer-Aided Design*. IEEE, 45–52. https://doi.org/10.1109/fmcad.2009.5351142

[17] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *Formal Methods and Software Engineering: 23rd International Conference on Formal Engineering Methods, ICFEM 2022, Madrid, Spain, October 24–27, 2022, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 90–105. https://doi.org/10.1007/978-3-031-17244-1_6

[18] Thomas Dinsdale-Young, Lars Birkedal, Philippa Gardner, Matthew Parkinson, and Hongseok Yang. 2013. Views: compositional reasoning for concurrent programs. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Rome, Italy) *(POPL '13)*. Association for Computing Machinery, New York, NY, USA, 287–300. https://doi.org/10.1145/2429069.2429104

[19] Azadeh Farzan, Dominik Klumpp, and Andreas Podelski. 2022. Sound sequentialization for concurrent program verification. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) *(PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 506–521. https://doi.org/10.1145/3519939.3523727

[20] Cormac Flanagan and Shaz Qadeer. 2003. Types for atomicity. In *Proceedings of the 2003 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation* (New Orleans, Louisiana, USA) *(TLDI '03)*. Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/604174.604176

[21] Robert W. Floyd. 1993. *Assigning Meanings to Programs*. Springer Netherlands, Dordrecht, 65–81. https://doi.org/10.1007/978-94-011-1793-7_4

[22] Matthew Fluet, Greg Morrisett, and Amal Ahmed. 2006. Linear Regions Are All You Need. In *Programming Languages and Systems*, Peter Sestoft (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 7–21.

[23] Aymeric Fromherz, Aseem Rastogi, Nikhil Swamy, Sydney Gibson, Guido Martínez, Denis Merigoux, and Tahina Ramananandro. 2021. Steel: proof-oriented programming in a dependently typed concurrent separation logic. *Proc. ACM Program. Lang.* 5, ICFP, Article 85 (Aug. 2021), 30 pages. https://doi.org/10.1145/3473590

[24] Jean-Yves Girard. 1987. Linear Logic. *Theoretical Computer Science* 50 (1987), 1–102.

[25] J.-Y. Girard. 1995. Linear Logic: its syntax and semantics. In *Advances in Linear Logic* (1 ed.), Jean-Yves Girard, Yves Lafont, and Laurent Regnier (Eds.). Cambridge University Press, 1–42. https://doi.org/10.1017/CBO9780511629150.002

[26] Léon Gondelman, Simon Oddershede Gregersen, Abel Nieto, Amin Timany, and Lars Birkedal. 2021. Distributed causal memory: modular specification and verification in higher-order distributed separation logic. *Proc. ACM Program. Lang.* 5, POPL, Article 42 (Jan. 2021), 29 pages. https://doi.org/10.1145/3434323

[27] Léon Gondelman, Jonas Kastberg Hinrichsen, Mário Pereira, Amin Timany, and Lars Birkedal. 2023. Verifying Reliable Network Components in a Distributed Separation Logic with Dependent Separation Protocols. *Proc. ACM Program. Lang.* 7, ICFP, Article 217 (Aug. 2023), 31 pages. https://doi.org/10.1145/3607859

[28] Ekanshdeep Gupta, Nisarg Patel, and Thomas Wies. 2025. Raven: An SMT-Based Concurrency Verifier. In *Computer Aided Verification (Lecture Notes in Computer Science)*. Springer Nature Switzerland, Cham, 81–106. https://doi.org/10.1007/978-3-031-98668-0_4

[29] Travis Hance, Yi Zhou, Andrea Lattuada, Reto Achermann, Alex Conway, Ryan Stutsman, Gerd Zellweger, Chris Hawblitzel, Jon Howell, and Bryan Parno. 2023. Sharding the State Machine: Automated Modular Reasoning for Complex Concurrent Systems. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 911–929. https://www.usenix.org/conference/osdi23/presentation/hance

[30] Chris Hawblitzel, Erez Petrank, Shaz Qadeer, and Serdar Tasiran. 2015. Automated and Modular Refinement Reasoning for Concurrent Programs. In *Computer Aided Verification*, Daniel Kroening and Corina S. Păsăreanu (Eds.). Springer International Publishing, Cham, 449–465.

[31] Stefan Heule, K. Rustan M. Leino, Peter Müller, and Alexander J. Summers. 2013. Abstract Read Permissions: Fractional Permissions without the Fractions. In *Verification, Model Checking, and Abstract Interpretation*, Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 315–334.

[32] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust verification by functional translation. *Proc. ACM Program. Lang.* 6, ICFP, Article 116 (Aug. 2022), 31 pages. https://doi.org/10.1145/3547647

[33] C. A. R. Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (Oct. 1969), 576–580. https://doi.org/10.1145/363235.363259

[34] Samin S. Ishtiaq and Peter W. O'Hearn. 2001. BI as an assertion language for mutable data structures. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (London, United Kingdom) *(POPL '01)*. Association for Computing Machinery, New York, NY, USA, 14–26. https://doi.org/10.1145/360204.375719

[35] Bart Jacobs, Dragan Bosnacki, and Ruurd Kuiper. 2018. Modular Termination Verification of Single-Threaded and Multithreaded Programs. *ACM Trans. Program. Lang. Syst.* 40, 3, Article 12 (July 2018), 59 pages. https://doi.org/10.1145/3210258

[36] Bart Jacobs, Jan Smans, and Frank Piessens. 2010. A Quick Tour of the VeriFast Program Verifier. In *Programming Languages and Systems*, Kazunori Ueda (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 304–311.

[37] Cliff B. Jones. 1983. Tentative Steps Toward a Development Method for Interfering Programs. *ACM Transactions on Programming Languages and Systems* 5, 4 (1983), 596–619. https://doi.org/10.1145/69575.69577

[38] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.* 2, POPL, Article 66 (Dec. 2017), 34 pages. https://doi.org/10.1145/3158154

[39] Ralf Jung, Robbert Krebbers, Lars Birkedal, and Derek Dreyer. 2016. Higher-order ghost state. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming* (Nara, Japan) *(ICFP 2016)*. Association for Computing Machinery, New York, NY, USA, 256–269. https://doi.org/10.1145/2951913.2951943

[40] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Mumbai, India) *(POPL '15)*. Association for Computing Machinery, New York, NY, USA, 637–650. https://doi.org/10.1145/2676726.2676980

[41] Johannes Kloos, Rupak Majumdar, and Viktor Vafeiadis. 2015. Asynchronous Liquid Separation Types. In *29th European Conference on Object-Oriented Programming (ECOOP 2015) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 37)*, John Tang Boyland (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 396–420. https://doi.org/10.4230/LIPIcs.ECOOP.2015.396

[42] Bernhard Kragl, Constantin Enea, Thomas A. Henzinger, Suha Orhun Mutluergil, and Shaz Qadeer. 2020. Inductive sequentialization of asynchronous programs. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 227–242. https://doi.org/10.1145/3385412.3385980

[43] Bernhard Kragl and Shaz Qadeer. 2018. Layered Concurrent Programs. In *Computer Aided Verification*, Hana Chockler and Georg Weissenbacher (Eds.). Springer International Publishing, Cham, 79–102.

[44] Bernhard Kragl and Shaz Qadeer. 2021. The Civl Verifier. In *Formal Methods in Computer Aided Design*. IEEE, 143–152.

[45] Bernhard Kragl, Shaz Qadeer, and Thomas A. Henzinger. 2020. Refinement for Structured Concurrent Programs. In *Computer Aided Verification*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 275–298.

[46] Shuvendu K. Lahiri, Shaz Qadeer, and David Walker. 2011. Linear maps. In *Proceedings of the 5th ACM Workshop Programming Languages meets Program Verification, PLPV 2011, Austin, TX, USA, January 29, 2011*, Ranjit Jhala and Wouter Swierstra (Eds.). ACM, 3–14. https://doi.org/10.1145/1929529.1929531

[47] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. Verus: A Practical Foundation for Systems Verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 438–454. https://doi.org/10.1145/3694715.3695952

[48] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 85 (April 2023), 30 pages. https://doi.org/10.1145/3586037

[49] K. Rustan M. Leino. 2010. Dafny: an automatic program verifier for functional correctness. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning* (Dakar, Senegal) *(LPAR'10)*. Springer-Verlag, Berlin, Heidelberg, 348–370.

[50] K. Rustan M. Leino, Peter Müller, and Jan Smans. 2009. *Verification of Concurrent Programs with Chalice*. Springer Berlin Heidelberg, Berlin, Heidelberg, 195–222. https://doi.org/10.1007/978-3-642-03829-7_7

[51] Jialin Li, Andrea Lattuada, Yi Zhou, Jonathan Cameron, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2022. Linear types for large-scale systems verification. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 69 (April 2022), 28 pages. https://doi.org/10.1145/3527313

[52] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-Based Verification for Rust Programs. In *Programming Languages and Systems*, Peter Müller (Ed.). Springer International Publishing, Cham, 484–514.

[53] Kenneth L. McMillan and Oded Padon. 2020. Ivy: A Multi-modal Verification Tool for Distributed Algorithms. In *Computer Aided Verification (Lecture Notes in Computer Science, Vol. 12225)*. Springer, 190–202. https://doi.org/10.1007/

978-3-030-53291-8_12

[54] Greg Morrisett, Amal Ahmed, and Matthew Fluet. 2005. L3: A Linear Language with Locations. In *Typed Lambda Calculi and Applications*, Paweł Urzyczyn (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 293–307.

[55] Ike Mulder, Robbert Krebbers, and Herman Geuvers. 2022. Diaframe: automated verification of fine-grained concurrent programs in Iris. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) *(PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 809–824. https://doi.org/10.1145/3519939.3523432

[56] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. 2016. Viper: A Verification Infrastructure for Permission-Based Reasoning. In *Proceedings of the 17th International Conference on Verification, Model Checking, and Abstract Interpretation - Volume 9583* (St. Petersburg, FL, USA) *(VMCAI 2016)*. Springer-Verlag, Berlin, Heidelberg, 41–62. https://doi.org/10.1007/978-3-662-49122-5_2

[57] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and separation in hoare type theory. In *Proceedings of the Eleventh ACM SIGPLAN International Conference on Functional Programming* (Portland, Oregon, USA) *(ICFP '06)*. Association for Computing Machinery, New York, NY, USA, 62–73. https://doi.org/10.1145/1159803.1159812

[58] Aleksandar Nanevski, Viktor Vafeiadis, and Josh Berdine. 2010. Structuring the verification of heap-manipulating programs. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Madrid, Spain) *(POPL '10)*. Association for Computing Machinery, New York, NY, USA, 261–274. https://doi.org/10.1145/1706299.1706331

[59] Abel Nieto, Léon Gondelman, Alban Reynaud, Amin Timany, and Lars Birkedal. 2022. Modular verification of op-based CRDTs in separation logic. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 188 (Oct. 2022), 29 pages. https://doi.org/10.1145/3563351

[60] Peter O'Hearn, John Reynolds, and Hongseok Yang. 2001. Local Reasoning about Programs that Alter Data Structures. In *Computer Science Logic*, Laurent Fribourg (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–19.

[61] Peter W. O'Hearn. 2004. Resources, Concurrency and Local Reasoning. In *CONCUR 2004 - Concurrency Theory*, Philippa Gardner and Nobuko Yoshida (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 49–67.

[62] Susan Owicki and David Gries. 1976. Verifying Properties of Parallel Programs: An Axiomatic Approach. *Commun. ACM* 19, 5 (1976), 279–285. https://doi.org/10.1145/360051.360224

[63] François Pottier and Jonathan Protzenko. 2013. Programming with permissions in Mezzo. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming* (Boston, Massachusetts, USA) *(ICFP '13)*. Association for Computing Machinery, New York, NY, USA, 173–184. https://doi.org/10.1145/2500365.2500598

[64] Patrick Redmond, Gan Shen, Niki Vazou, and Lindsey Kuper. 2023. Verified Causal Broadcast with Liquid Haskell. In *Proceedings of the 34th Symposium on Implementation and Application of Functional Languages* (Copenhagen, Denmark) *(IFL '22)*. Association for Computing Machinery, New York, NY, USA, Article 6, 13 pages. https://doi.org/10.1145/3587216.3587222

[65] J.C. Reynolds. 2002. Separation logic: a logic for shared mutable data structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. 55–74. https://doi.org/10.1109/LICS.2002.1029817

[66] Patrick M. Rondon, Ming Kawaguci, and Ranjit Jhala. 2008. Liquid types. *SIGPLAN Not.* 43, 6 (June 2008), 159–169. https://doi.org/10.1145/1379022.1375602

[67] Ilya Sergey, Aleksandar Nanevski, and Anindya Banerjee. 2015. Mechanized verification of fine-grained concurrent programs. *SIGPLAN Not.* 50, 6 (June 2015), 77–87. https://doi.org/10.1145/2813885.2737964

[68] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, Frans Kaashoek, and Nickolai Zeldovich. 2023. Grove: a Separation-Logic Library for Verifying Distributed Systems. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 113–129. https://doi.org/10.1145/3600006.3613172

[69] Jan Smans, Bart Jacobs, and Frank Piessens. 2012. Implicit dynamic frames. *ACM Trans. Program. Lang. Syst.* 34, 1, Article 2 (May 2012), 58 pages. https://doi.org/10.1145/2160910.2160911

[70] Frederick Smith, David Walker, and J. Gregory Morrisett. 2000. Alias Types. In *Proceedings of the 9th European Symposium on Programming Languages and Systems (ESOP '00)*. Springer-Verlag, Berlin, Heidelberg, 366–381.

[71] Amin Timany, Simon Oddershede Gregersen, Léo Stefanesco, Jonas Kastberg Hinrichsen, Léon Gondelman, Abel Nieto, and Lars Birkedal. 2024. Trillium: Higher-Order Concurrent and Distributed Separation Logic for Intensional Refinement. *Proc. ACM Program. Lang.* 8, POPL, Article 9 (Jan. 2024), 32 pages. https://doi.org/10.1145/3632851

[72] Mads Tofte and Jean-Pierre Talpin. 1994. Implementation of the typed call-by-value λ-calculus using a stack of regions. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon, USA) *(POPL '94)*. Association for Computing Machinery, New York, NY, USA, 188–201. https://doi.org/10.1145/174675.177855

[73] Moshe Y. Vardi. 1991. Verification of concurrent programs: the automata-theoretic framework*. *Annals of Pure and Applied Logic* 51, 1 (1991), 79–98. https://doi.org/10.1016/0168-0072(91)90066-U

[74] Philip Wadler. 1990. Linear Types can Change the World!. In *Programming Concepts and Methods*. https://api.semanticscholar.org/CorpusID:58535510

[75] Philip Wadler. 2012. Propositions as sessions. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming* (Copenhagen, Denmark) *(ICFP '12)*. Association for Computing Machinery, New York, NY, USA, 273–286. https://doi.org/10.1145/2364527.2364568

[76] Christopher Wagner, Nouraldin Jaber, and Roopsha Samanta. 2023. Enabling Bounded Verification of Doubly-Unbounded Distributed Agreement-Based Systems via Bounded Regions. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 81 (April 2023), 29 pages. https://doi.org/10.1145/3586033

[77] David Walker and J. Gregory Morrisett. 2000. Alias Types for Recursive Data Structures. In *Selected Papers from the Third International Workshop on Types in Compilation (TIC '00)*. Springer-Verlag, Berlin, Heidelberg, 177–206.

[78] David Walker and Kevin Watkins. 2001. On regions and linear types (extended abstract). In *Proceedings of the Sixth ACM SIGPLAN International Conference on Functional Programming* (Florence, Italy) *(ICFP '01)*. Association for Computing Machinery, New York, NY, USA, 181–192. https://doi.org/10.1145/507635.507658